MICROSOFT CERTIFIED · ASSOCIATE · EXAM DP 700

Fabric Data Engineer Associate

The complete study course for Implementing Data Engineering Solutions Using Microsoft Fabric, built objective by objective from the official skills outline.

EXAM TIP

How Microsoft tends to frame a question, and the keyword in the scenario that points to the correct answer.

EXAM TRAP

A plausible distractor that candidates often pick. Learn why it is wrong as well as why the right answer is right.

REMEMBER

A fact, default value or limit worth memorising word for word before exam day.

CHECKPOINT

A scenario question at the end of every objective group, with the answer and reasoning revealed below it.

Each objective flows: Concept → Diagram → Syntax → Tips → Checkpoint

The exam at a glance

100
minutes of seat time for the questions

700
scaled score out of 1000 to pass

3
domains, each weighted 30 to 35%

12
months valid, then a free online renewal

Item	Detail
Full title	Implementing Data Engineering Solutions Using Microsoft Fabric
Credential earned	Microsoft Certified: Fabric Data Engineer Associate
Delivery	Pearson VUE, online proctored or at a test centre
Fee	Varies by country (USD 165 in the United States); check the exam page
Item count	Not published; expect roughly 40 to 60 items, possibly with a case study

REMEMBER Most questions cover generally available features, but commonly used preview features can appear.

ORIENTATION · DOMAIN WEIGHTING

Three domains, three equal thirds



Dark segment: guaranteed minimum of 30%. Light segment: the band up to 35% in which the exact share varies between exam forms.

EXAM TIP No domain can be skipped. A weak domain costs about a third of the paper, so plan revision time in equal blocks and hunt down your weakest area first.

ORIENTATION · OBJECTIVE MAP 1 OF 3

Domain 1: Implement and manage an analytics solution

Configure workspace settings

Spark settings · Domain settings · OneLake settings · Apache Airflow settings

Implement lifecycle management

Version control · Database projects · Deployment pipelines

Configure security and governance

Workspace and item access · Row, column, object, folder and file access · Dynamic data masking · Sensitivity labels · Endorsement · Audit logs · OneLake security

Orchestrate processes

Choose Dataflow Gen2, pipeline or notebook · Schedules and event based triggers · Orchestration patterns with parameters and dynamic expressions

REMEMBER Covered in slides 21 to 93 of this deck. Workspace settings changed in the October 2026 revision, so study the Airflow and OneLake settings closely.

Domain 2: Ingest and transform data

Loading patterns

- Full and incremental loads
- Prepare data for a dimensional model
- Loading pattern for streaming data

Batch data

- Choose a data store
- Choose Dataflows Gen2, notebooks, KQL or T SQL
- OneLake shortcuts
- Mirroring
- Ingest with pipelines
- Transform with PySpark, SQL, KQL
- Denormalize, group, aggregate
- Duplicate, missing and late data

Streaming data

- Choose a streaming engine
- Native tables or shortcuts in Real Time Intelligence
- Query acceleration or standard shortcuts
- Eventstream
- Spark structured streaming
- KQL
- Windowing functions

REMEMBER Covered in slides 94 to 153. This is the most syntax heavy domain: expect code completion items in PySpark, T SQL and KQL.

Domain 3: Monitor and optimize an analytics solution

Monitor Fabric items

- Data ingestion
- Data transformation
- Semantic model refresh
- Configure alerts

Identify and resolve errors

- Pipeline errors
- Dataflow Gen2 errors
- Notebook errors
- Eventhouse errors
- Eventstream errors
- T SQL errors
- OneLake shortcut errors

Optimize performance

- Lakehouse tables
- Pipelines
- Data warehouse
- Eventstream and Eventhouse
- Spark performance
- Query performance

REMEMBER Covered in slides 154 to 186. Slides 187 to 196 then pull everything together into exam strategy, cheat sheets and a final checklist.

ORIENTATION · AUDIENCE PROFILE

Who the exam is written for

You bring subject matter expertise in **data loading patterns, data architectures and orchestration**. Your three core responsibilities mirror the three domains:

1 · Ingesting and transforming data

2 · Securing and managing an analytics solution

3 · Monitoring and optimizing an analytics solution

You work closely with analytics engineers, architects, analysts and administrators.

Three languages you must read and write

SQL
T SQL in the Warehouse, SQL analytics endpoint and SQL database; Spark SQL in notebooks.

PySpark
DataFrame API, Delta Lake operations and structured streaming in notebooks.

KQL
Kusto Query Language in Eventhouse KQL databases and querysets.

Question formats you should expect

Multiple choice

One correct answer, or several where the stem says “select all that apply” or states how many.

Drag and drop

Order the steps of a process, or match tools to requirements. Order often matters for partial credit.

Code completion

Pick the missing keyword in PySpark, T SQL or KQL from drop down lists inside a code block.

Hot area

Select options in a screenshot or table, such as a setting in a workspace pane.

Yes or No series

The same scenario with a proposed solution per screen. You cannot return to earlier screens.

Case study

A company story with tabs for requirements and environment, then a block of linked questions.

EXAM TRAP In a Yes or No series, answer each screen on its own merits. More than one proposed solution can meet the goal.

The October 2026 skills revision

Skill area	Domain	Change
Audience profile	All	No change
Configure Microsoft Fabric workspace settings	Domain 1	Minor
Other Domain 1 groups	Domain 1	No change
Ingest and transform data	Domain 2	No change
Optimize performance	Domain 3	Minor
Other Domain 3 groups	Domain 3	No change

REMEMBER The current workspace settings list names Spark, domain, OneLake and Apache Airflow settings. Older courses may miss Airflow.

EXAM TIP Optimize performance now lists Eventstream and Eventhouse together. Know their policies and throughput options.

ORIENTATION · STUDY PLAN

A six week plan that covers every objective

W1 — Foundations

Fabric, OneLake, Delta, capacities, workspace settings. Build a trial workspace.

W2 — Lifecycle and security

Git, database projects, deployment pipelines, roles, RLS, CLS, masking, labels.

W3 — Orchestration and loads

Pipelines, expressions, triggers, incremental loads, SCD, star schemas.

W4 — Batch and streaming

Shortcuts, mirroring, PySpark, T SQL, KQL, Eventstream, windows.

W5 — Monitor and optimize

Monitor hub, errors, V Order, OPTIMIZE, Spark tuning, Eventhouse policies.

W6 — Exam rehearsal

Free practice assessment, exam sandbox, weak area repair, book the exam.

EXAM TIP Hands on time is the best predictor of a pass. A free Fabric trial capacity lets you run every lab in this deck.

ORIENTATION · ROADMAP

The journey through this deck

Slides 1 to 20

Orientation and foundations

Exam facts, Fabric, OneLake, Delta, medallion, items.

Slides 21 to 93

Domain 1 · Implement and manage

Settings, lifecycle, security, orchestration.

Slides 94 to 153

Domain 2 · Ingest and transform

Loading patterns, batch, streaming.

Slides 154 to 186

Domain 3 · Monitor and optimize

Monitoring, troubleshooting, performance.

Slides 187 to 196

Exam readiness

Strategy, traps, cheat sheets, glossary.

Throughout

11 checkpoints

One scenario quiz closes every objective group.

Microsoft Fabric in one picture

Governance and security: Microsoft Purview, sensitivity labels, endorsement, audit — spans everything

Data Factory	Data Engineering	Data Warehouse	Real Time Intelligence
Databases	Data Science	Power BI	Industry solutions

OneLake · one logical data lake per tenant · Delta Parquet as the shared format

Fabric capacity (F SKU) supplies the compute units every workload draws on

REMEMBER Fabric is software as a service: one tenant, one OneLake, one capacity model, one security and governance layer across every engine.

The workloads a data engineer touches

Workload	Key items	Language	Typical use
Data Factory	Pipeline, Dataflow Gen2, Copy job, Mirroring	Expressions, M	Ingest, move, orchestrate
Data Engineering	Lakehouse, Notebook, Spark job definition, Environment	PySpark, Spark SQL	Large scale transformation
Data Warehouse	Warehouse, SQL analytics endpoint	T SQL	Relational modelling and serving
Real Time Intelligence	Eventstream, Eventhouse, KQL database, Activator	KQL	Streaming and telemetry
Databases	SQL database in Fabric	T SQL	Operational, transactional data
Power BI	Semantic model, report	DAX	Consumption and Direct Lake
Orchestration	Apache Airflow job	Python DAGs	Code first workflows

EXAM TIP When a scenario names the team's skill set, map it straight to a tool: SQL skills point to Warehouse, Python or Spark skills to notebooks, low code skills to Dataflow Gen2, telemetry to KQL.

Capacities, SKUs and capacity units

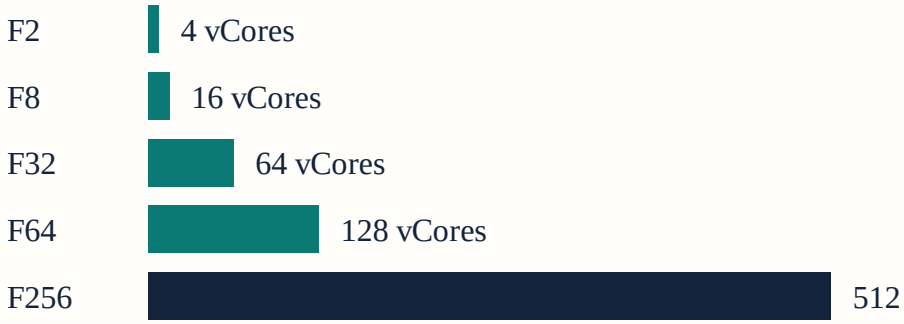
A **Fabric capacity** is a pool of compute measured in **capacity units (CU)**. Workspaces are assigned to a capacity, and every item they hold draws CUs from it.

F SKUs run from F2 to F2048; the number is the CU count.

1 CU = 2 Spark vCores for Spark sizing.

Bursting lets Spark queue up to 3 times the base vCores.

Pause and resume on pay as you go capacities stop billing.



REMEMBER A Fabric trial gives a capacity equivalent to F64 for evaluation, which is ample for practising every objective.

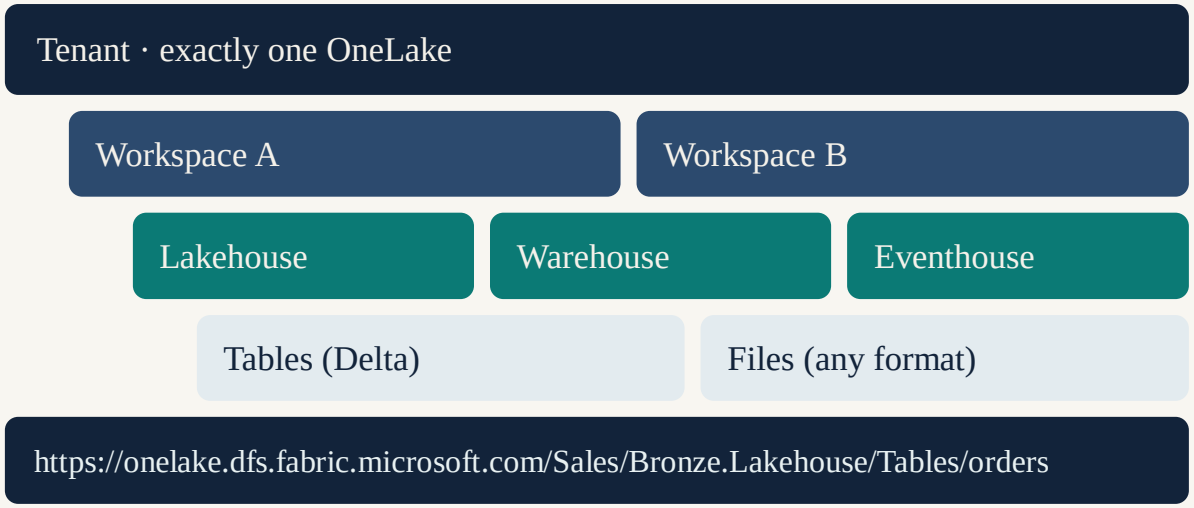
What happens when a capacity is overloaded

Fabric **smooths** usage over time: interactive operations over at least 5 minutes, background operations (pipelines, Spark, refreshes) over 24 hours. Throttling then escalates in stages based on how much future capacity is already consumed.

Up to 10 minutes	10 to 60 minutes	60 minutes to 24 hours	Over 24 hours
Overage protection	**Interactive delay**	**Interactive rejection**	**Background rejection**
No throttling. Work runs on borrowed future capacity.	Interactive requests are delayed by 20 seconds.	Interactive requests are rejected; background jobs still run.	All requests, including scheduled jobs, are rejected.

EXAM TIP Use the Fabric Capacity Metrics app to see utilisation, overages and throttling. Fixes: scale up the SKU, reschedule heavy jobs, or optimise the expensive items.

OneLake: one lake, one copy, every engine



Built on ADLS Gen2, so tools that speak the ADLS APIs and the ABFS driver can read and write it.

One copy: Spark, T SQL, KQL and Power BI read the same Delta files without duplication.

Shortcuts and mirroring bring external data in without building copy pipelines.

Delta Lake: Parquet files plus a transaction log

Tables/orders/

_delta_log/

00000000000000000000.json

00000000000000000001.json

00000000000000000010.checkpoint.parquet

part 00000 … .snappy.parquet

part 00001 … .snappy.parquet

Each commit adds a JSON entry that records which Parquet files were added or removed. Readers trust the log, not the folder listing.

ACID

Atomic commits; readers never see half written data.

Time travel

Query earlier versions with VERSION AS OF or TIMESTAMP AS OF.

Schema enforcement

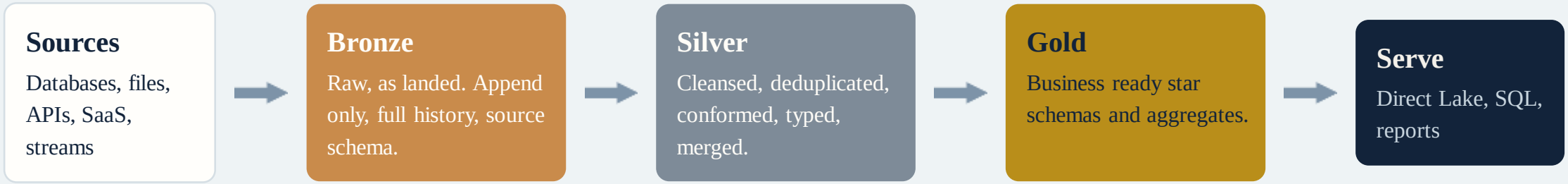
Mismatched writes fail unless you allow evolution with mergeSchema.

DML

UPDATE, DELETE and MERGE on a data lake.

REMEMBER Only Delta tables in the Tables area are discovered as tables and exposed through the SQL analytics endpoint.

The medallion architecture



Common Fabric layout: one lakehouse per layer, or one workspace per layer for stronger security isolation.

Gold store choice: a lakehouse for Spark teams, a Warehouse for T SQL teams needing multi table transactions.

EXAM TIP Deduplication and data quality rules belong in Silver. Dimensional modelling and aggregates belong in Gold.

The core items and what each one is for

Item	What it is	Write with
Lakehouse	Delta tables plus files, with an automatic SQL analytics endpoint	Spark, pipelines, Dataflow Gen2
SQL analytics endpoint	Read only T SQL view over lakehouse Delta tables	Views, functions only
Warehouse	Fully transactional T SQL warehouse on Delta storage	T SQL, pipelines, Dataflow Gen2
Eventhouse	Container for KQL databases, tuned for time series	Eventstream, KQL, pipelines
Mirrored database	Near real time replica of an external database	Mirroring engine (read only)
SQL database	Operational Azure SQL engine inside Fabric	T SQL applications
Notebook and Spark job definition	Interactive or batch Spark code	PySpark, Scala, R, SQL
Pipeline, Dataflow Gen2, Copy job	Orchestration, low code transformation, simple copy	Designer, Power Query
Eventstream and Activator	Stream routing and event driven actions	No code designer

EXAM TRAP The SQL analytics endpoint cannot INSERT, UPDATE or DELETE lakehouse tables. Write with Spark, or use a Warehouse.

DOMAIN 1 · 30 TO 35% OF THE EXAM

Implement and manage an analytics solution

01

- Configure workspace settings
- Implement lifecycle management
- Configure security and governance
- Orchestrate processes

Structure of a Fabric workspace

Workspace

The unit of collaboration, security, capacity assignment and deployment.

- Items
- Roles
- Capacity
- Domain
- Git branch
- Pipeline stage

Workspace settings panes you must know

Pane	Configures
License info	Capacity assignment and SKU
Domain	Which domain the workspace belongs to
Git integration	Repo, branch and folder connection
OneLake	Shortcut caching and OneLake options
Data Engineering/Science	Spark pools, environment, jobs
Apache Airflow settings	Airflow runtime pool and nodes
Workspace identity	Managed identity for trusted access
Monitoring	Workspace monitoring Eventhouse

The Spark settings tabs at a glance

Pool	Default pool for the workspace (Starter or a custom pool), node family and size, autoscale, dynamic executor allocation, and whether items may customise compute.
Environment	Spark runtime version and the default environment item that supplies libraries and Spark properties to every notebook and job.
High concurrency	Allow notebooks, and notebooks run by pipelines, to share one Spark session.
Automatic log	Automatic MLflow logging of experiments and models.
Jobs	Session idle timeout and job admission behaviour, such as reserving maximum cores for active jobs.

REMEMBER You need the workspace Admin role to change workspace Spark settings.

Starter pool or custom pool?

Starter pool

- Prehydrated, always warm clusters
- Sessions start in seconds
- Medium nodes only
- You can tune max nodes and executors
- Best for interactive development

Custom pool

- You pick node size and node count
- Autoscale between min and max nodes
- Cold start, typically a few minutes
- Sized to a workload such as large joins
- Capacity admins can create capacity pools

EXAM TIP “Reduce session start time” points to the starter pool. “Needs large memory nodes” points to a custom pool.

EXAM TRAP Installing custom libraries in an environment forces a fresh session, so starts slow even on the starter pool.

Node sizes for custom pools

Size	vCores	Memory
Small	4	32 GB
Medium	8	64 GB
Large	16	128 GB
X Large	32	256 GB
XX Large	64	512 GB



REMEMBER A pool cannot use more vCores than its capacity allows. On an F8 (16 vCores, 48 with bursting), a pool of large nodes fits only a few nodes.

Autoscale and dynamic allocation

Autoscale (nodes)

The pool grows and shrinks the number of **nodes** between a minimum and maximum as demand changes.



Node count over a job’s lifetime

Dynamic allocation (executors)

Spark adds and removes **executors** for a session within min and max limits, releasing idle resources to other jobs.

Driver | Executor 1 | Executor 2 | Executor 3 (released)

EXAM TIP Autoscale answers “vary cluster size with load”. Dynamic allocation answers “share executors between concurrent sessions fairly”. Both are pool level toggles.

Environments: packaged Spark configuration

Environment item

- Spark runtime version
- Public libraries (PyPI, Conda)
- Custom libraries (.whl, .jar)
- Spark properties
- Compute (pool and sizing)
- Resources (small files)

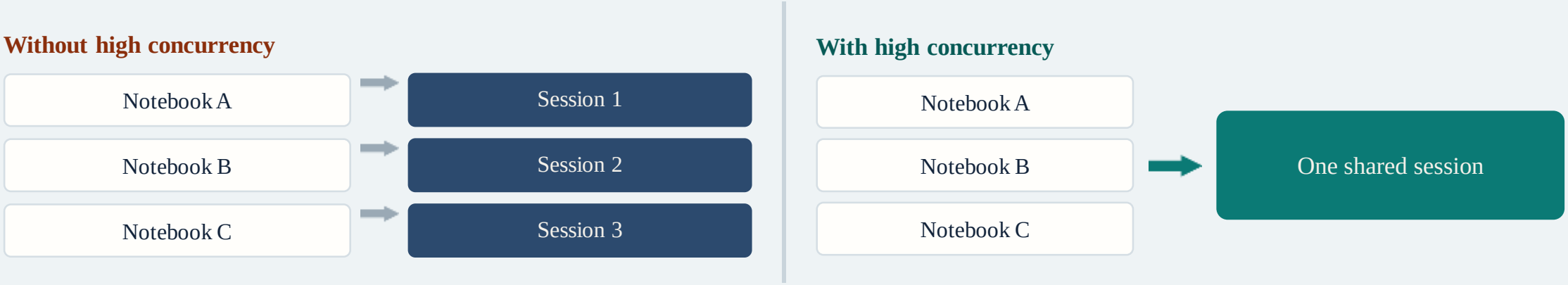
Attach to a notebook or Spark job definition, or set it as the **workspace default**.

Publish after changes; library changes take effect only once published.

Source control: environments are Git supported items, so configuration moves between stages.

EXAM TIP “Every notebook in the workspace must use the same library versions” is solved by a workspace default environment, not by %pip install in each notebook.

High concurrency mode: share one session



Conditions: same user, same environment, same Spark configuration. Each notebook keeps its own isolated REPL.

Pipelines: enable the pipeline option and give notebook activities the same session tag to reuse a session.

EXAM TIP A pipeline running many notebooks in sequence that wastes time on start ups: enable high concurrency for pipelines.

Runtimes and the native execution engine

Fabric Spark runtimes

Runtime	Spark	Delta Lake
1.2	3.4	2.4
1.3	3.5	3.2
2.0 (newer)	4.x	4.x

Set the runtime at workspace level or per environment. Check Microsoft Learn for the runtimes in support on your exam date.

Native execution engine

A vectorised C++ engine (Velox with Apache Gluten) that runs Spark SQL and DataFrame operators natively, with automatic fallback to the JVM for anything unsupported.

Enable it in an environment, or per session with the Spark property:

```
spark.native.enabled = true
```

No code changes and no extra cost.

EXAM TIP “Speed up existing Spark SQL over Parquet and Delta without rewriting code” points to the native execution engine.

Jobs settings and item level compute control

Session timeout

Idle interactive sessions end automatically, 20 minutes by default. Shorten it to free capacity sooner; lengthen it for long debugging work.

Job admission

By default Fabric admits a job on its minimum cores (optimistic admission). Reserving maximum cores guarantees full scale out but admits fewer concurrent jobs.

Customize compute for items

When on, users can change pool and executor sizing per environment or session. Turn it off to force every item onto the workspace default pool.

Queueing

Batch jobs from pipelines and schedules queue when capacity is busy. Interactive notebook sessions are rejected rather than queued.

EXAM TRAP Governance asks such as “stop developers creating oversized sessions” are met by disabling item level compute customisation, not by removing their workspace role.

Three places to set a Spark property

1 · Environment (widest)

Spark properties tab. Applies to every item using the environment, including the workspace default.

2 · Session start

The %%configure magic in the first cell sets driver and executor sizing and conf for that session.

3 · Runtime (narrowest)

spark.conf.set in code, for SQL settings that can change while the session runs.

```
%%configure
{
  "driverMemory": "28g", "driverCores": 4,
  "executorMemory": "28g", "executorCores": 4,
  "conf": {"spark.sql.shuffle.partitions": "200"}
}
```

```
spark.conf.set("spark.sql.shuffle.partitions", "64")
spark.conf.set(
  "spark.sql.autoBroadcastJoinThreshold", "104857600")
spark.conf.get("spark.sql.shuffle.partitions")
```

REMEMBER %%configure must run before the session starts; with the force flag it restarts the session. Cluster level settings cannot change through spark.conf.set.

Domains: organizing Fabric by business area

Tenant (Fabric admin creates domains)

Domain: Sales

- Subdomain: EMEA
- Subdomain: Americas

Workspaces: Sales Bronze, Sales Gold

Domain: Finance

- Subdomain: Payroll
- Subdomain: Audit

Workspaces: Ledger, Reporting

Why: supports a data mesh with federated ownership, and filters the OneLake catalog so users find their area’s data.

Rule: a workspace belongs to at most one domain or subdomain at a time.

Domain roles and domain settings

Role	What they can do
Fabric admin	Create and delete domains, assign domain admins, bulk assign workspaces by name, owner or capacity
Domain admin	Edit the domain, create subdomains, assign contributors, set default domain, override delegated tenant settings
Domain contributor	Assign workspaces they administer to the domain (they must also be workspace Admin)
Workspace admin	Choose the domain in the workspace settings Domain pane, if they are a contributor

Default domain

Users or groups whose new workspaces land in the domain automatically.

Delegated settings

Tenant settings a domain admin may override, such as default sensitivity label and certification.

Subdomains

One extra level for finer grouping inside a domain.

OneLake settings: tenant and workspace levels

Tenant level (admin portal)

- Users can access OneLake data with apps external to Fabric (ADLS APIs, Azure Storage Explorer, Databricks)
- Users can sync data with OneLake file explorer
- Short lived user delegation SAS tokens

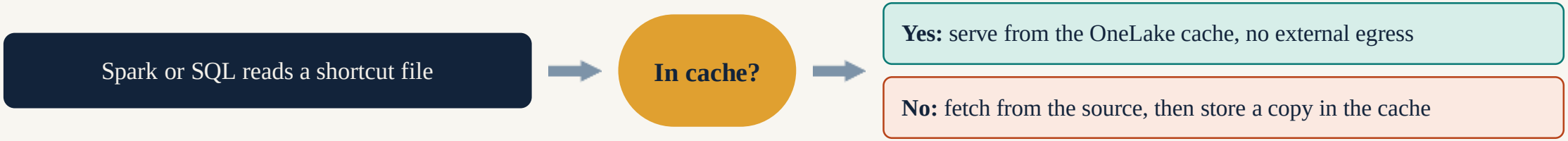
Workspace level (OneLake pane)

- Enable cache for shortcuts and set its retention period
- Reset the shortcut cache
- OneLake diagnostics: capture data access events into a lakehouse for auditing

EXAM TIP “External tools cannot reach OneLake” is a tenant setting problem, not a workspace role problem.

REMEMBER Workspace admins configure the OneLake pane; tenant toggles belong to Fabric administrators.

Shortcut caching cuts egress and latency



Sources cached

Amazon S3, S3 compatible, Google Cloud Storage and on premises gateway shortcuts.

Retention

1 to 28 days; the clock resets each time a cached file is read.

Limits

Individual files over 1 GB are not cached. Changed source files are fetched fresh.

EXAM TIP “Reduce cloud egress charges from repeated reads of S3 data” is answered by enabling shortcut caching in workspace settings.

Apache Airflow jobs in Fabric

An **Apache Airflow job** is a Fabric item that hosts a managed Airflow runtime. You write workflows as Python **DAGs** (directed acyclic graphs) instead of drawing them in the pipeline canvas.

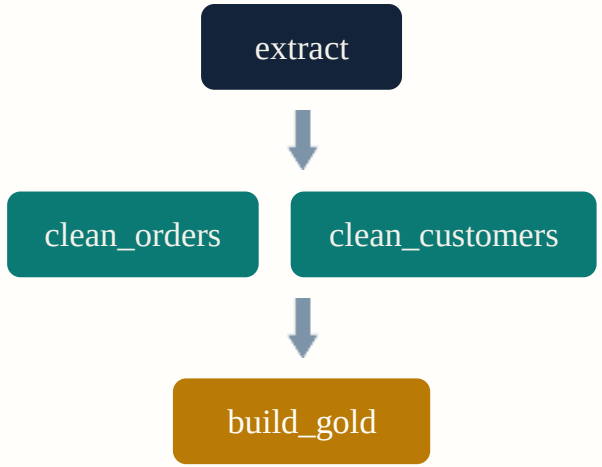
DAG storage: Fabric managed storage or Git sync

Packages: extra Python requirements per job

Config: Airflow configuration overrides and environment variables

Reach: run Fabric notebooks and pipelines, plus hundreds of community operators

A simple DAG



EXAM TIP Choose an Airflow job when the team already writes Airflow DAGs in Python or needs code first orchestration across many systems.

Configuring the Airflow runtime for a workspace

Setting	Purpose
Default pool	Starter pool or a custom pool for all Airflow jobs
Starter pool	Fast start; pauses when idle, so suits development
Custom pool	Always on, so scheduled DAGs fire on time in production
Compute node size	Small or Large, by DAG complexity and parallel tasks
Autoscale	Adds workers when many tasks queue
Extra nodes	More concurrent tasks for heavy schedules

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG("daily_sales",
  start_date=datetime(2026, 1, 1),
  schedule="@daily", catchup=False) as dag:
  extract = BashOperator(task_id="extract",
    bash_command="echo extract")
  load = BashOperator(task_id="load",
    bash_command="echo load")
  extract >> load
```

EXAM TRAP A starter pool that pauses when idle is a poor fit for strict production schedules. Pick a custom, always on pool there.

CHECKPOINT 1 · WORKSPACE SETTINGS

Test yourself: three quick scenarios

Q1. Analysts wait minutes for each new notebook session. Libraries are standard. What do you change?

Answer: Use the starter pool as the workspace default; avoid custom libraries that force a new cluster.

Q2. New workspaces created by the HR team must appear under the HR domain automatically. Who configures what?

Answer: A domain admin sets the HR team as a default domain group.

Q3. Daily notebooks reread the same Google Cloud Storage files and egress costs keep rising. What helps most?

Answer: Enable shortcut caching in the workspace OneLake settings with a suitable retention.

The two pillars of Fabric lifecycle management

Git integration

Version control: item definitions committed to Azure DevOps or GitHub, branching, history, pull requests.

Source controls metadata, never data rows

Deployment pipelines

Promote content between workspaces representing stages: Development, Test, Production.

Every stage is a separate workspace

Database projects

Warehouse and SQL database schema as code, built to a DACPAC and deployed declaratively.

Rules and variables change per stage

EXAM TIP Git answers “track changes, collaborate, roll back”. Deployment pipelines answer “promote to test and production with different connections”.

Connecting a workspace to Git

1

Admin enables Git integration in tenant settings

2

Workspace Admin opens Workspace settings, Git integration

3

Choose Azure DevOps or GitHub and authenticate

4

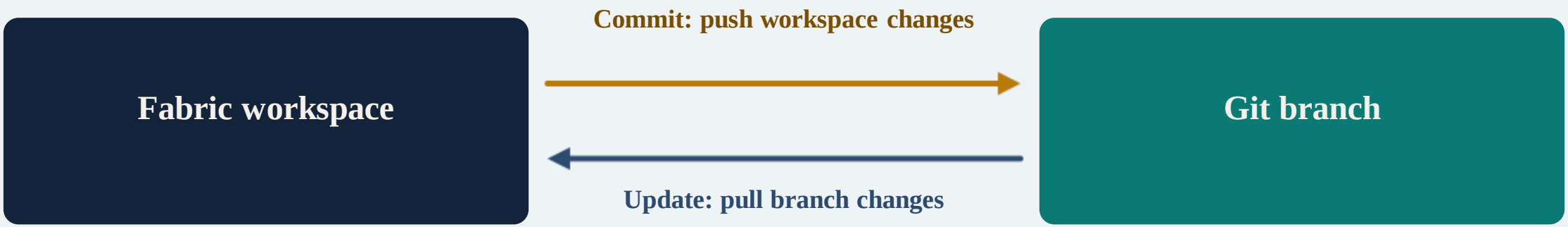
Pick organisation, project, repo, branch and folder

5

Initial sync: workspace to Git, or Git to workspace

Fact	Detail
Providers	Azure DevOps (Azure Repos) and GitHub, including GitHub Enterprise
One to one	A workspace connects to exactly one branch and folder at a time
Who can connect	Workspace Admin; Contributors and above can commit and update
Format	Each item becomes a folder of definition files (JSON, .py, .sql, platform file)

Commit and update: the two way sync



Synced: item matches the branch

Uncommitted: edited in the workspace

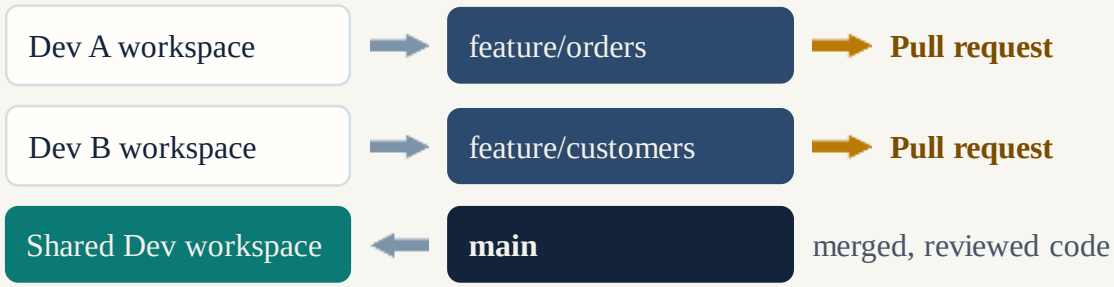
Update required: changed in Git

Conflict: changed on both sides

REMEMBER Commits can be selective, item by item, with a message. Update pulls every incoming change at once, and is blocked while conflicts remain.

DOMAIN 1 · VERSION CONTROL

Branching: one workspace per branch



Branch out creates a new branch and a new connected workspace in one step from the source control pane.

Isolation stops developers overwriting each other in a shared workspace.

Reviews happen in the Git provider through pull requests and branch policies.

EXAM TIP “Several engineers must work on the same items without interfering” is answered with feature branches, each connected to its own workspace.

Resolving conflicts and undoing work

Resolve in Fabric

In the source control pane, choose per item: **accept incoming** (Git wins) or **keep current** (workspace wins), then commit.

Resolve in Git

Check out a new branch from the workspace state, merge in the provider, then **switch branch** or update the workspace.

Undo

Discards uncommitted workspace changes and returns items to the last synced state.

Typical flow for a conflict

Update fails → Open conflicts list → Pick a version per item → Update completes → Commit

EXAM TRAP Disconnecting and reconnecting Git does not merge anything; it only resets the link. Resolve conflicts explicitly.

DOMAIN 1 · VERSION CONTROL

What Git stores, and what it never stores

Item definitions in source control

Notebook | Pipeline | Dataflow Gen2 | Environment | Spark job definition | Lakehouse (metadata) | Warehouse (schema) | SQL database | Eventhouse and KQL database | Eventstream | Semantic model | Report | Variable library | Copy job | Mirrored database

Supported item types grow often; check the current list on Microsoft Learn.

Never in Git

- Table data and files in OneLake
- Credentials and connection secrets
- Workspace role assignments
- Run history and refresh history

EXAM TRAP Syncing a lakehouse to a new workspace creates an empty lakehouse. Data must be reloaded or reached through shortcuts.

Database projects: schema as code

A **SQL database project** (.sqlproj) holds one .sql file per object: tables, views, functions, stored procedures, security. It describes the **desired state** of a Warehouse or SQL database.

The build produces a **.dacpac**. Deploying it compares the DACPAC with the target and generates only the changes needed.

> **REMEMBER** Fabric Git integration already serialises a Warehouse and a SQL database as a database project.

SalesDW.sqlproj

- dbo/Tables/DimCustomer.sql
- dbo/Tables/FactSales.sql
- dbo/Views/vSalesByMonth.sql
- dbo/StoredProcedures/usp_LoadSales.sql
- Security/rls_SalesFilter.sql

```
CREATE TABLE dbo.DimCustomer (
  CustomerKey INT NOT NULL,
  CustomerName VARCHAR(100),
  City VARCHAR(50)
);
```

The database project workflow

Extract
Create the project from an existing Warehouse, or start empty

Edit
VS Code with the SQL Database Projects extension

Build
Validates references and produces the DACPAC

Commit
Push to Git; a pull request triggers CI

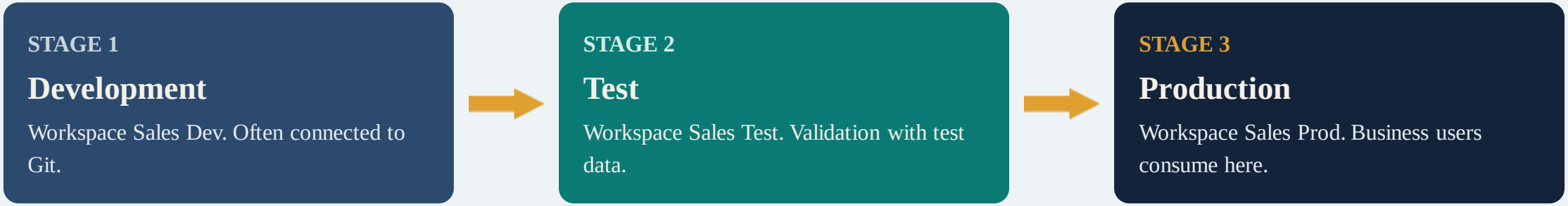
Publish
SqlPackage deploys the DACPAC to the target

```
dotnet build SalesDW.sqlproj
SqlPackage /Action:Publish /SourceFile:bin/Debug/SalesDW.dacpac /TargetConnectionString:"Server=<warehouse endpoint>;Database=SalesDW"
```

EXAM TIP Use a database project when schema changes must be reviewed, built and deployed repeatably.

EXAM TRAP A DACPAC holds schema, not data. Data loads still run in pipelines or procedures.

Deployment pipelines: promote through stages



2 to 10 stages, named as you wish, fixed once created.

One workspace is assigned per stage, and a workspace belongs to only one pipeline.

Content, not data, is copied; target data stays untouched.

REMEMBER Deploying a lakehouse creates or keeps its structure in the target, never its rows. Pipelines and notebooks keep their definitions.

How a deployment actually works

Item pairing

Items with the same name and type in adjacent stages are paired. Paired items are overwritten on deploy; unpaired items are created.

Compare

Shows new, different and missing items between stages, with a change review before you deploy.

Selective deployment

Deploy only chosen items; “select related” adds dependencies such as the lakehouse a notebook uses.

Backward deployment

Deploy to an earlier stage, for example to sync a hotfix from Production back to Test. Allowed only to an empty target or for new items.

EXAM TIP Renaming an item breaks pairing, so the next deployment creates a duplicate. Keep names identical across stages.

Deployment rules: same item, different settings

Test stage notebook

Default lakehouse: **Sales_Test**



Production notebook after rule

Default lakehouse: **Sales_Prod**

Rule type	Changes	Typical items
Data source rule	Server, database or connection a model points to	Semantic models, dataflows
Parameter rule	Value of a defined parameter	Semantic models, dataflows
Default lakehouse rule	Lakehouse a notebook attaches to	Notebooks

REMEMBER Rules are set on the **target** stage, after the item has been deployed there once, and apply from the next deployment.

EXAM TIP For pipelines and notebooks, variable libraries now handle many per stage values more flexibly than rules.

Who can do what in a deployment pipeline

Action	Permission needed
Create a deployment pipeline	Any licensed user; creator becomes pipeline admin
Assign a workspace to a stage	Pipeline admin plus workspace Admin of that workspace
View the pipeline and compare	Pipeline admin (sharing grants access to the pipeline only)
Deploy to a stage	Pipeline admin plus Contributor or higher in source and target workspaces
Configure deployment rules	Pipeline admin plus owner of the item in the target stage
Delete the pipeline	Pipeline admin

EXAM TRAP Pipeline access does not grant workspace access. A tester also needs a role in the Test workspace.

EXAM TIP Least privilege for a release engineer: pipeline admin plus Contributor on each stage workspace.

Variable libraries: values that follow the stage

A **variable library** is a workspace item holding named variables, each with a default value and optional **value sets**. Each stage marks one value set as active.

Consumed by pipelines, notebooks and other supported items

Types include string, integer, boolean, datetime and item reference

Source controlled and deployable like any item

Variable	Default	Test	Prod
source_server	dev01	test01	prod01
batch_size	100	1000	50000
send_alerts	false	false	true

Active value set per stage: Dev uses Default, Test uses Test, Prod uses Prod.

EXAM TIP “Avoid editing the pipeline after each deployment to change connection values” points to a variable library with a value set per stage.

Automating CI/CD for Fabric

Fabric REST APIs

Git APIs (connect, commit, update from Git) and Deployment Pipelines APIs (deploy stage content) let you script every manual step.

fabric_cicd Python library

Open source library that deploys item definitions from a repo into a workspace, with parameter files for per environment values.

Azure Pipelines or GitHub Actions

Run the APIs or the library on pull request merge, with approvals and gates between stages.

Service principal identity

Automation authenticates as a service principal or managed identity, allowed by a tenant setting and given workspace roles.

REMEMBER Long running API calls return 202 Accepted with an operation location you poll until the operation succeeds or fails.

Four documented CI/CD patterns

Pattern	How it works	Choose when
1 · Git based, branch per stage	Dev, Test and Prod each map to a branch; merges promote; each workspace updates from its branch	Git is the single source of truth
2 · Git based with build environments	One main branch; a build step transforms definitions per stage and deploys with APIs	Stage values must be rewritten in CI
3 · Git plus deployment pipelines	Only Dev links to Git; deployment pipelines promote Dev to Test to Prod	Simple, UI driven releases
4 · ISV multi tenant	Templates deployed into many customer workspaces through APIs	Software vendors

EXAM TIP Pattern 3 is the most common exam answer: Git on Dev only, deployment pipelines for promotion, rules or variable libraries for stage values.

EXAM TRAP Connecting Prod to the same branch as Dev pushes untested work straight to users.

CHECKPOINT 2 · LIFECYCLE MANAGEMENT

Test yourself: lifecycle scenarios

Q1. After deploying to Production, notebooks still read the Test lakehouse. Fastest fix without editing code?

Answer: Add a default lakehouse deployment rule on the Production stage, then redeploy.

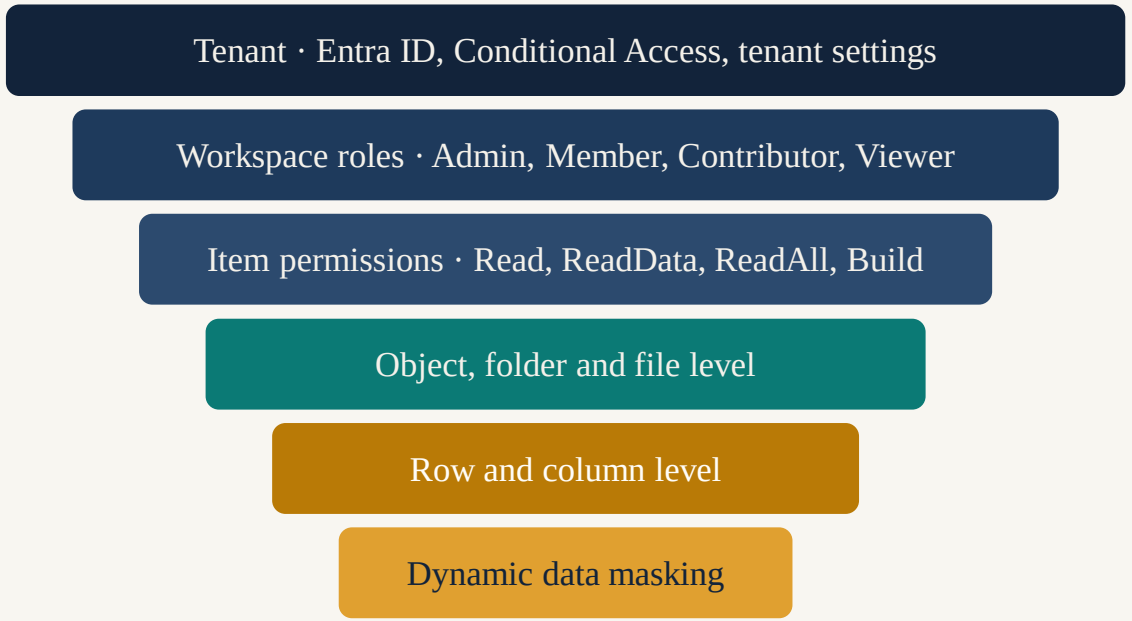
Q2. Warehouse schema changes must be peer reviewed and deployed as incremental ALTER scripts. What do you use?

Answer: A SQL database project in Git, built to a DACPAC and published with SqlPackage.

Q3. Two engineers keep overwriting each other's notebook edits in one Dev workspace. What do you recommend?

Answer: Feature branches, each branched out to its own workspace, merged by pull request.

Security in layers, from tenant to cell



Broad to narrow. Each layer can only restrict what the layer above allows.

Governance alongside: sensitivity labels, endorsement, audit logs, OneLake security.

EXAM TIP Always pick the least privileged option that meets the requirement.

The workspace role matrix

Capability	Admin	Member	Contributor	Viewer
Delete workspace, manage Admins, change settings	Yes	No	No	No
Add Members, Contributors, Viewers	Yes	Yes	No	No
Share items and grant reshare	Yes	Yes	No	No
Create, edit and delete items	Yes	Yes	Yes	No
Run notebooks, pipelines, Spark jobs	Yes	Yes	Yes	No
Read and write OneLake data directly	Yes	Yes	Yes	No
Commit and update with Git	Yes	Yes	Yes	No
Query Warehouse and SQL endpoint (T SQL)	Yes	Yes	Yes	Yes
View items and reports	Yes	Yes	Yes	Yes

EXAM TIP Contributor is the least privileged role that can build and run things. Viewer is read only and cannot read lakehouse files through Spark.

Assigning roles the right way

Use groups

Assign roles to Microsoft Entra security groups or Microsoft 365 groups, not individuals. Joiners and leavers then need no workspace changes.

Highest role wins

A user in two groups with different roles gets the more permissive role. Item permissions add to, never remove, a role.

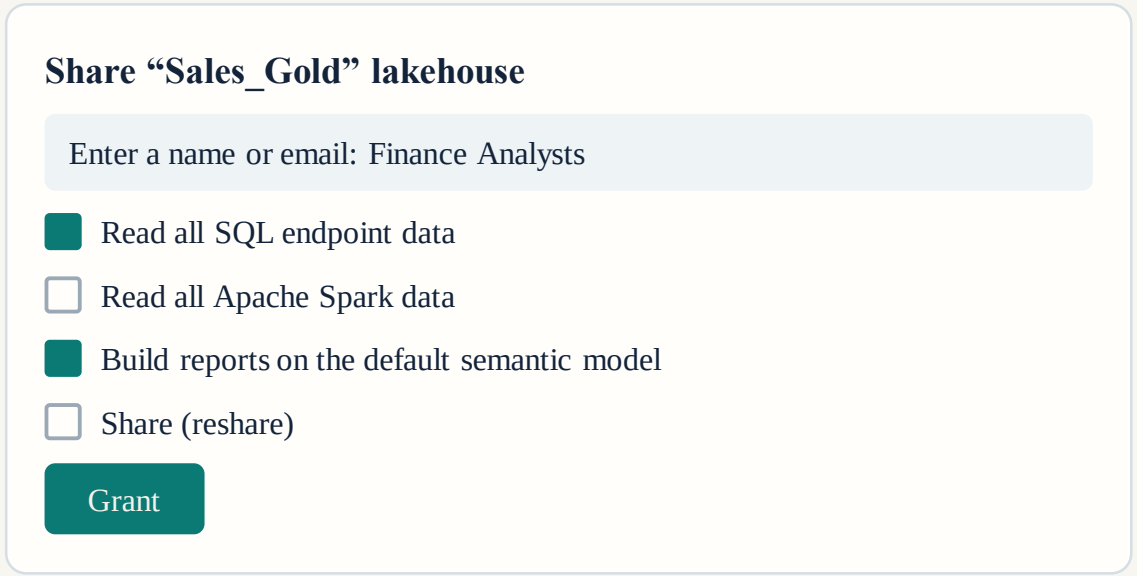
Separate by layer

Put Bronze, Silver and Gold in different workspaces so analysts get Viewer on Gold only.

Where to manage Workspace, Manage access, Add people or groups, choose the role. Admins and Members can do this; Members cannot add Admins.

EXAM TRAP Row level security in a Warehouse does not restrict Admin, Member or Contributor users reading files directly in OneLake. Give consumers Viewer or item access only.

Sharing an item without a workspace role



Sharing always grants **Read** (connect and see metadata). Each tick adds one more permission.

REMEMBER Read alone lets a user connect to the SQL endpoint but query nothing until granted ReadData or object level GRANTs.

EXAM TIP Sharing one item is how you give access to that item only, rather than every item in the workspace.

Item permissions decoded

Permission	Share dialog wording	What it allows
Read	Granted by any share	See the item and connect; metadata only
ReadData	Read all SQL endpoint data	Query every table through T SQL, subject to SQL security
ReadAll	Read all Apache Spark / OneLake data	Read files and tables directly in OneLake with Spark or APIs
Build	Build reports on semantic model	Create reports and composite models
Write	Workspace role only	Change the item definition or data
Reshare	Share	Pass on the same permissions to others
Execute	Workspace role or API	Run an item such as a pipeline or notebook

EXAM TRAP ReadAll bypasses SQL row level security, because Spark reads the files directly.

EXAM TIP SQL analysts with RLS: grant ReadData only. Data scientists using Spark: grant ReadAll.

Which engine enforces which control

Control	Warehouse and SQL endpoint	OneLake security	Semantic model and KQL
Row level	Security policy with predicate function	Row filter in a data access role	DAX roles; KQL row level security policy
Column level	GRANT SELECT on named columns	Column selection in a role	Object level security hides columns
Object level	GRANT or DENY on tables, views, procedures, schemas	Table selection in a role	Object level security hides tables
Folder and file	Not applicable	Folder selection in the Files area	Not applicable
Masking	Dynamic data masking	Not applicable	Not applicable

REMEMBER T SQL security applies only to queries through the SQL engine. OneLake security is defined once at the data and enforced across Spark, SQL and Power BI.

Row level security: same query, different rows

dbo.Sales (all rows)

SalesRep	Region	Amount
ana@contoso.com	North	1,200
ben@contoso.com	South	900
ana@contoso.com	North	450
ben@contoso.com	South	2,100



Ana runs SELECT * FROM dbo.Sales and sees 2 rows: 1,200 and 450.

Ben runs the same query and sees 2 rows: 900 and 2,100.

The **filter predicate** silently adds WHERE SalesRep = USER_NAME() to every read.

EXAM TIP Fabric Warehouse supports **filter** predicates for RLS. Block predicates, which stop writes, are not supported.

Implementing RLS in T SQL, step by step

Step 1 · Schema for security objects

```
CREATE SCHEMA Security;
```

Step 2 · Inline table valued predicate function

```
CREATE FUNCTION Security.fn_SalesRep(@SalesRep AS VARCHAR(100))
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS fn_result
WHERE @SalesRep = USER_NAME()
  OR USER_NAME() = 'manager@contoso.com';
```

Step 3 · Security policy that binds it

```
CREATE SECURITY POLICY Security.SalesFilter
ADD FILTER PREDICATE Security.fn_SalesRep(SalesRep)
ON dbo.Sales WITH (STATE = ON);
```

SCHEMABINDING is required so the referenced objects cannot change underneath the policy.

USER_NAME() returns the caller’s Entra principal name in Fabric.

STATE = OFF disables the policy without dropping it.

REMEMBER Function returns a table; policy uses ADD FILTER PREDICATE ... ON table.

Column level security with GRANT

dbo.Employees

EmpID	Name	Dept	Salary	NI No

Teal: granted to HR Analysts. Rust: no grant, so hidden.

```
GRANT SELECT ON dbo.Employees
  (EmpID, Name, Dept) TO [HR Analysts];

DENY SELECT ON dbo.Employees (Salary)
  TO [Contractors];
```

Behaviour: SELECT EmpID, Name works. SELECT * fails with a permission error because it touches Salary.

DENY beats GRANT when a user belongs to groups with both.

EXAM TIP If users must still run SELECT * but see obscured values, the answer is dynamic data masking, not column level security.

Object level security on tables, views and schemas

Grant a whole schema

```
GRANT SELECT ON SCHEMA::gold TO [Analysts];
```

Hide one table

```
DENY SELECT ON gold.Payroll TO [Analysts];
```

Expose data only through a view

```
CREATE VIEW gold.vSalesSummary AS
SELECT Region, SUM(Amount) AS Total
FROM gold.Sales GROUP BY Region;
GRANT SELECT ON gold.vSalesSummary TO [Execs];
```

Allow a procedure, not the tables

```
GRANT EXECUTE ON dbo.usp_LoadSales TO [Loaders];
```

Ownership chaining lets a view or procedure read tables the caller cannot read directly, when owners match.

Semantic models use object level security to hide tables and columns from report users.

REMEMBER Users need item Read (connect) before any T SQL GRANT matters.

OneLake security: define once, enforce everywhere

OneLake security uses **roles** defined on a lakehouse (and other supported items) that grant access to specific tables or folders, optionally with row and column restrictions.

Open the item, choose **Manage OneLake security**, create a role, pick the data, add members.

> **EXAM TRAP** Roles restrict only users without write access. Admin, Member and Contributor bypass OneLake security roles.

Anatomy of a role

Name	EMEA_Sales_Readers
Data	Tables/sales, Tables/customers
Rows	WHERE Region = 'EMEA'
Columns	All except Margin
Members	EMEA Analysts group

A default reader role, granting everything to users with item read access, exists until you edit or remove it.

How OneLake security reaches every engine



Spark notebooks
Reads return only permitted tables, rows and columns

SQL endpoint
Honours roles when set to user identity access mode

Power BI Direct Lake
Models on OneLake use the viewer’s identity

Shortcuts
Internal shortcuts carry the target’s roles with them

EXAM TIP “One security definition that applies to Spark, SQL and Power BI alike” points to OneLake security rather than T SQL RLS.

Folder level access in the Files area

Lakehouse: Raw_Landing

Files/

crm/ (role: CRM_Team)

erp/ (role: Finance_Team)

hr/ (role: HR_Team only)

Tables/

orders (role: All_Analysts)

A role covering a folder covers all its subfolders and files.

Use case: raw files from different source systems land in one lakehouse, but each team sees only its own folder.

Members need item Read on the lakehouse and membership of the folder's role.

Access paths: Spark, OneLake file explorer and ADLS compatible APIs all respect the role.

EXAM TIP File level access is achieved by folder roles; there is no per file ACL in the exam scope.

Decision guide: requirement to control

“Users see only their own region’s rows”	Row level security
“Salary column must be completely unavailable”	Column level security
“Show emails as aXXX@XXXX.com to support staff”	Dynamic data masking
“Hide the Payroll table from analysts”	Object level security (DENY)
“Team sees only its folder of raw files”	OneLake security folder role
“Access one lakehouse, nothing else in the workspace”	Item sharing

Dynamic data masking: obscure, do not remove

Stored value (privileged user)

Email: jane.smith@contoso.com

Phone: 07700 900123

Card: 4111 1111 1111 1234

Salary: 52000



Query result (masked user)

Email: jXXX@XXXX.com

Phone: xxxx

Card: XXXX XXXX XXXX 1234

Salary: 0

Applied at query time by the SQL engine. The data on disk is unchanged.

Who sees real values: Admin, Member, Contributor, and anyone granted UNMASK.

EXAM TRAP Masking is not encryption. Users with ad hoc query rights can infer values with WHERE filters. Pair it with RLS, CLS or OLS for true protection.

The four masking functions

Function	Behaviour	Example output
default()	Full mask by data type: strings XXXX, numbers 0, dates 1 January 1900	XXXX · 0
email()	Keeps the first letter, constant .com suffix	jXXX@XXXX.com
random(1, 100)	Numeric columns only; random value in the range	57
partial(2, "XXXX", 2)	Keeps a prefix and suffix, pads the middle with your string	07XXXX23

Names, notes
default()

Email address
email()

Discount, score
random()

Card, phone
partial()

EXAM TIP “Show the last four digits of a card number” is always partial(0, "XXXX XXXX XXXX ", 4).

Masking syntax and the UNMASK permission

Mask at creation

```
CREATE TABLE dbo.Customers (
  CustomerID INT,
  Email VARCHAR(100) MASKED WITH (FUNCTION = 'email()'),
  Phone VARCHAR(20) MASKED WITH (FUNCTION = 'partial(2,"XXXX",2)')
);
```

Add or remove a mask later

```
ALTER TABLE dbo.Customers ALTER COLUMN CreditLimit
  ADD MASKED WITH (FUNCTION = 'random(1000, 5000)');
ALTER TABLE dbo.Customers ALTER COLUMN Phone DROP MASKED;
```

Let a group see real values

```
GRANT UNMASK TO [Fraud Analysts];
REVOKE UNMASK FROM [Fraud Analysts];
```

Where: Warehouse and the lakehouse SQL analytics endpoint.

Granularity: UNMASK can be granted at database, schema, table or column level.

REMEMBER Keyword order: MASKED WITH (FUNCTION = '...').

Sensitivity labels from Microsoft Purview

Public	General	Confidential	Highly Confidential

Labels and their order are defined by the organisation in Purview; this sequence is a common example, least to most sensitive.

Apply manually from the item header or settings by anyone with write access (Contributor or above).

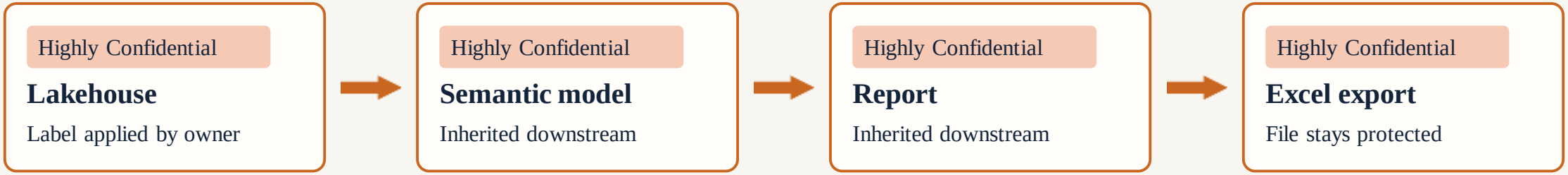
Default label policy stamps new items automatically; a **mandatory** policy forces users to choose one.

Protection travels: exports to Excel, PowerPoint or PDF keep the label and its encryption.

Prerequisite: the Fabric admin enables information protection tenant settings.

EXAM TRAP A sensitivity label classifies and protects exported content. It does not restrict who can query the data inside Fabric.

Label inheritance down the lineage



Downstream inheritance pushes a label to dependent items when the new label is more sensitive, and never overwrites a manually set label.

Inheritance from data sources lets a model pick up the label of supported Azure sources it connects to.

REMEMBER Lineage view shows which items carry which label, and is the fastest way to audit label coverage across a workspace.

DOMAIN 1 · ENDORSEMENT

Endorsement: signalling trustworthy content

Promoted

Who: any user with write access to the item.

Means: “I think this is ready to be shared and reused.”

Certified

Who: reviewers authorised by the Fabric admin or domain admin.

Means: meets the organisation’s quality standards.

Master data

Who: designated reviewers set by the admin.

Means: the authoritative single source for core data such as customers or products.

EXAM TIP Master data applies to data holding items such as lakehouses, warehouses and semantic models, not to reports.

REMEMBER An item carries one endorsement at a time. Endorsed items rank higher in the OneLake catalog.

DOMAIN 1 · AUDIT LOGS

Fabric audit logs: who did what, and when

Fabric activity
View, create, edit, delete, share, export, run

→

Unified audit log
Microsoft Purview, tenant wide

→

Ways to read it
Purview portal Audit search · Office 365 Management Activity API · PowerShell · Admin monitoring workspace

Who can search: users with the Audit Logs or View Only Audit Logs role in Purview, or Fabric administrators.

Filter by date range, user, activity (for example ViewArtifact, ShareArtifact) and workload.

SQL audit logs for a Warehouse record T SQL actions such as logins and queries, written to OneLake for analysis.

Retention depends on the Purview licence; export logs you must keep longer.

EXAM TIP “Identify which user exported data from a report last week” is answered by searching the Purview unified audit log.

Audit, diagnostics or monitoring?

Tool	Scope	Captures	Question it answers
Purview audit log	Tenant	User and admin activities on items	Who shared, exported or deleted?
Warehouse SQL audit logs	One warehouse	T SQL events and logins	Who queried this table?
OneLake diagnostics	Workspace	Data access events on OneLake	Which files were read, by whom?
Workspace monitoring	Workspace	Operational logs and metrics in an Eventhouse	Why was this query or refresh slow?
Capacity Metrics app	Capacity	CU consumption and throttling	What is overloading the capacity?

REMEMBER Audit is about accountability. Monitoring is about performance and health.

EXAM TRAP The Monitor hub shows run status, not security events. It is not an audit tool.

Test yourself: security scenarios

Q1. Analysts query a Warehouse in T SQL. Each must see only their department's rows. Least effort?

Answer: Predicate function plus CREATE SECURITY POLICY with a filter predicate.

Q2. Support agents may see only the last four digits of phone numbers, yet fraud analysts need full values.

Answer: partial() mask on Phone, then GRANT UNMASK to the fraud analyst group.

Q3. A data scientist needs Spark access to one lakehouse only, without seeing other workspace items.

Answer: Share the lakehouse with "Read all Apache Spark data" (ReadAll).

Q4. The company wants one lakehouse flagged as the authoritative customer source.

Answer: Endorse it as Master data through a designated reviewer.

Orchestration: deciding what runs, when, and in what order

What

Choose the right engine: Dataflow Gen2, pipeline, notebook, or a combination.

When

Schedules for time based runs; event based triggers for file arrival, job events and data changes.

How

Patterns: parent and child pipelines, metadata driven loops, parameters and dynamic expressions.

A typical daily run

Schedule 02:00 → Copy to Bronze → Notebook to Silver → Procedure to Gold → Refresh model

EXAM TIP The pipeline is the usual conductor: it calls notebooks, dataflows, procedures and refreshes, and handles dependencies, retries and alerts.

Dataflow Gen2, pipeline or notebook?

Criterion	Dataflow Gen2	Pipeline	Notebook
Primary job	Low code transformation	Orchestration and copy	Code first transformation
Language	Power Query M (visual)	Activities and expressions	PySpark, Spark SQL, Scala, R
Skill set	Analysts, Power BI users	Integration engineers	Data engineers, scientists
Data volume	Small to medium	Any (moves data)	Large to very large
Connectors	Hundreds of Power Query sources	Copy activity connectors, gateways	Spark readers, libraries, APIs
Control flow	None	Loops, conditions, retries	Python logic, notebookutils
Best for	Shaping and cleansing tables	Ingestion and scheduling	Complex logic, ML, merges

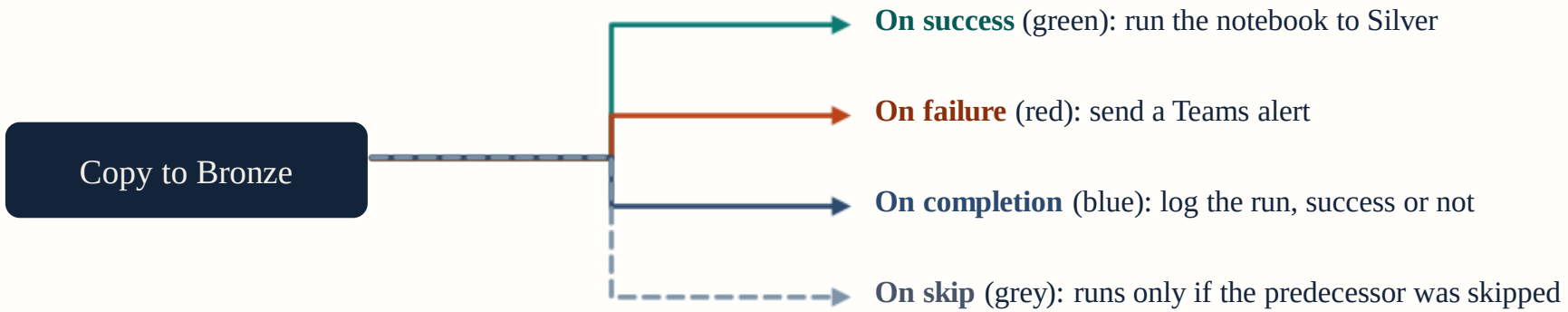
EXAM TIP Read the skill clue first: “no code” means Dataflow Gen2, “Python” or “large scale” means notebook, “orchestrate”, “copy” or “loop” means pipeline.

A decision tree for the exam

Do you need loops, conditions, scheduling or many steps chained?	→ Yes	Pipeline
Is it a straight copy of data with no transformation?	→ Yes	Copy activity or Copy job
Must non coders build the transformation visually?	→ Yes	Dataflow Gen2
Large volumes, complex logic, Delta MERGE, ML or custom libraries?	→ Yes	Notebook
Team writes Python DAGs or needs Airflow operators?	→ Yes	Apache Airflow job

REMEMBER These combine: a pipeline commonly orchestrates a Copy activity, then a notebook, then a Dataflow Gen2.

Activity dependencies: the four conditions



REMEMBER Several inputs into one activity are combined with AND: all must meet their conditions.

EXAM TRAP Handling a failure path makes the pipeline report success unless you add a Fail activity.

Activity catalogue 1: doing the work

Copy data
Moves data between any supported source and sink

Notebook
Runs a notebook with base parameters

Dataflow
Refreshes a Dataflow Gen2, optionally with parameters

Stored procedure
Calls a procedure in a Warehouse or SQL database

Script
Runs ad hoc SQL statements, query or non query

Spark job definition
Runs a packaged batch or streaming Spark job

Semantic model refresh
Refreshes a Power BI model after the load

KQL
Runs a KQL query or control command on an Eventhouse

Delete data
Removes files or folders from a store

Web and Webhook
Calls REST endpoints; Webhook waits for a callback

Outlook and Teams
Sends email or chat notifications

Copy job and Airflow
Invoke a Copy job or Airflow job item

Activity catalogue 2: control flow

Activity	Purpose	Key output or setting
Lookup	Read a row or rows (config table, watermark)	output.firstRow or output.value
Get Metadata	File existence, size, child items, last modified	output.childItems, output.exists
ForEach	Loop over an array	Sequential flag, batch count up to 50
If Condition	Branch on a boolean expression	True and False activity sets
Switch	Branch on a value with several cases	Cases plus a default
Until	Loop until a condition is true	Timeout
Set / Append variable	Store values during a run	String, boolean, array; return value
Invoke pipeline	Call a child pipeline	Wait on completion, pass parameters
Wait and Fail	Pause; or end the run with a custom error	Seconds; error message and code

EXAM TRAP Lookup returns at most 5,000 rows (and a limited size). For bigger lists, use Get Metadata, a notebook, or paginate.

Designing schedules

Schedule settings

Repeat	By minute, hourly, daily, weekly or monthly
Times	One or more times per day, or every N minutes or hours
Start and end	Date range in which the schedule is active
Time zone	Choose explicitly to handle daylight saving
Multiple	An item can carry several schedules

Schedulable items: pipelines, notebooks, Spark job definitions, Dataflow Gen2, Copy jobs, semantic model refreshes.

EXAM TIP Schedule the parent pipeline, not each notebook. One schedule then controls order and dependencies.

EXAM TRAP Schedules fire even if the previous run is still going. Prevent overlaps with sensible intervals or concurrency checks.

Event based triggers: run when something happens

- OneLake events: file created or deleted
- Azure Blob Storage events
- Job events: item run succeeded or failed
- Workspace item events



Activator rule

Filters by event type, folder and file name pattern



Pipeline runs

With the event details available to expressions

```
@pipeline()?.TriggerEvent?.FileName    returns the file that fired the trigger
@pipeline()?.TriggerEvent?.FolderPath  returns its folder
```

EXAM TIP “Process each file as soon as it lands” means a storage event trigger, not a frequent schedule.

REMEMBER Create triggers from the pipeline’s Add trigger button or from the Real Time hub.

Parameters, variables and library variables

Parameters

Set when the run starts, read only during the run. Passed in by a trigger, a parent pipeline or a manual run.

```
@pipeline().parameters.TableName
```

Variables

Changed during the run with Set variable or Append variable. Types: String, Boolean, Array, Integer.

```
@variables('RowCount')
```

Library variables

Read from a variable library, so the value follows the deployment stage.

```
@pipeline().libraryVariables.Server
```

System variables worth knowing

@pipeline().RunId · @pipeline().Pipeline · @pipeline().TriggerTime · @pipeline().TriggerType · @pipeline().GroupId (workspace)

EXAM TRAP A variable cannot reference itself in Set variable. Use a second variable as a temporary holder to increment a counter.

Reading dynamic expressions

`@pipeline().parameters.Region`	Whole value is an expression; the result keeps its type.
`sales_@{pipeline().parameters.Region}.parquet`	String interpolation: @{ } embeds a value inside text.
`@activity('LookupWM').output.firstRow.LastLoad`	A previous activity's output; the name must match exactly.
`@item().SourceTable`	The current element inside a ForEach loop.
`@@literal`	Two @ signs escape a literal @ at the start of text.

EXAM TIP Code completion items love the path output.firstRow (single row Lookup) versus output.value (all rows). Know which the Lookup setting "First row only" produces.

Expression function cheat sheet

Strings

@concat('raw/', pipeline().parameters.Source)
@replace(item().Name, ' ', '_')
@toUpper('emea') · @substring('abcdef', 0, 3)

Dates

@utcNow() · @formatDateTime(utcNow(), 'yyyyMMdd')
@getPastTime(1, 'Day') · @startOfDay(utcNow())
@convertFromUtc(utcNow(), 'GMT Standard Time')

Logic

@if(equals(variables('Mode'), 'Full'), 'overwrite', 'append')
@and(greater(1, 0), not(empty(item())))
@coalesce(pipeline().parameters.Date, utcNow())

Collections and conversion

@length(activity('GetFiles').output.childItems)
@split('a,b,c', ',') · @first(variables('List'))
@int('42') · @string(1) · @json('{"k":1}')

REMEMBER Functions nest inside a single leading @. Do not repeat @ inside the expression: @concat('a', utcNow()) is correct.

The metadata driven ingestion pattern

Control table

Source	Target
dbo.Orders	orders
dbo.Customers	customers
dbo.Products	products



Lookup

First row only: off



ForEach

Items: @activity('Lookup').output.value

Copy: source @item().Source, sink @item().Target

Add a table by inserting a row, not by editing the pipeline.

Parallelism via ForEach batch count; sequential when order matters.

Extend the table with load type, watermark column and active flag.

EXAM TIP “Ingest 200 tables with minimal development effort” is the Lookup plus ForEach plus parameterised Copy pattern.

Parameterising notebooks

1 · Mark a parameters cell

```
run_date = "20260101"
source = "erp"
full_load = False
```

Cell menu, Toggle parameter cell. Defaults used for manual runs.

2 · Pass values from a pipeline

Notebook activity, Base parameters

run_date: @formatDateTime(utcNow(), 'yyyyMMdd')

source: @item().Source

full_load: (Bool) false

Fabric injects a new cell after the parameters cell that overrides the defaults.

3 · Return a value

```
notebookutils.notebook.exit(
  str(row_count))
```

Read it in the pipeline with @activity('Notebook1').output.result.exitValue

EXAM TRAP Without a cell marked as parameters, base parameters are ignored and the notebook runs with hard coded values.

Orchestrating notebooks from a notebook

Run one child, wait for its result

```
result = notebookutils.notebook.run(
  "Load_Silver_Orders",  # notebook name
  600,  # timeout in seconds
  {"run_date": "20260101"})  # parameters
print(result)  # the child's exit value
```

Shares the caller's Spark session

No start up cost per child, but children run one after another unless you use runMultiple.

Run many in parallel with dependencies

```
dag = {"activities": [
  {"name": "orders", "path": "Load_Orders"},
  {"name": "customers", "path": "Load_Customers"},
  {"name": "gold", "path": "Build_Gold",
   "dependencies": ["orders", "customers"]}],
 "concurrency": 4, "timeoutInSeconds": 3600}
notebookutils.notebook.runMultiple(dag)
```



> **EXAM TIP** mssparkutils is the older name for notebookutils; both appear in exam questions with the same methods.

Building resilient orchestration

Retry and timeout

Each activity has Retry, Retry interval and Timeout in its General tab. Ideal for transient network or throttling errors.

Try and catch

Upon failure path to an alert, then a Fail activity so the run still shows Failed in monitoring.

Parent and child pipelines

Invoke pipeline keeps modules small and reusable; the parent owns schedule and error policy.

Idempotent loads

MERGE or overwrite by partition so a rerun after failure never duplicates data.

```
try:
  df = spark.read.table("bronze.orders")
  df.write.mode("overwrite").saveAsTable("silver.orders")
except Exception as e:
  notebookutils.notebook.exit(f"FAILED: {e}")  # or raise to fail the activity
```

EXAM TRAP Exiting a notebook with a message marks the activity Succeeded. Raise the exception if the pipeline must see a failure.

Test yourself: orchestration scenarios

Q1. CSV files land in a lakehouse folder at random times and must be loaded within minutes.

Answer: A OneLake file created event trigger on the pipeline, filtered to the folder and .csv suffix.

Q2. Complete the expression to read a single row Lookup value named Max: @activity('LKP').output.____.Max

Answer: firstRow

Q3. Business analysts must clean an Excel extract with no code each week.

Answer: Dataflow Gen2, scheduled or called from a pipeline.

Q4. Five independent notebooks must finish before a sixth, in the least time.

Answer: notebookutils.notebook.runMultiple with a DAG where the sixth depends on the five.

DOMAIN 2 · 30 TO 35% OF THE EXAM

Ingest and transform data

02

- Loading patterns
- Batch data
- Streaming data

Three loading patterns you must design

A

Full and incremental

Reload everything, or move only what changed since the last run using watermarks, change data capture or change data feed.

B

Dimensional model

Shape Silver data into facts and dimensions with surrogate keys and slowly changing dimension handling.

C

Streaming

Continuous ingestion of events with low latency, landing in an Eventhouse, a lakehouse or both.

REMEMBER The exam tests design judgement: picking the pattern from data volume, change frequency, latency target and whether the source can reveal what changed.

Full load versus incremental load

Aspect	Full load	Incremental load
What moves	Every row, every run	Only new or changed rows
Write mode	Overwrite or truncate and load	Append, or MERGE (upsert)
Needs	Nothing from the source	A watermark column, CDC or change feed
Deletes	Handled automatically	Need soft delete flags or CDC
Cost and time	Grows with total size	Grows with change volume
Best for	Small tables, reference data, first load	Large, frequently changing tables

Rows moved per day, 10 million row table, 1% daily change



The high watermark pattern

1 · Lookup old
Read last watermark from the control table

→

2 · Lookup new
Read MAX(ModifiedDate) from the source

→

3 · Copy delta
Rows between old and new watermark

→

4 · Merge
Upsert into the Silver table

→

5 · Update
Save the new watermark, only on success

```
SELECT * FROM dbo.Orders
WHERE ModifiedDate > '@{activity('LookupOld').output.firstRow.Watermark}'
  AND ModifiedDate <= '@{activity('LookupNew').output.firstRow.NewWatermark}'
```

EXAM TIP Capturing the new watermark before copying closes the gap for rows that change during the copy.

EXAM TRAP A watermark column cannot detect hard deletes in the source.

Change data capture and Delta change data feed

Source side CDC

The source database records inserts, updates and deletes. Fabric reads it through mirroring, Copy job CDC mode, or Eventstream CDC connectors.

Captures deletes, unlike watermarks.

Delta change data feed (lakehouse side)

```
ALTER TABLE silver.orders SET TBLPROPERTIES
  (delta.enableChangeDataFeed = true);

changes = (spark.read.format("delta")
  .option("readChangeFeed", "true")
  .option("startingVersion", 12)
  .table("silver.orders"))
```

_change_type value	Meaning
insert	New row
update_preimage / update_postimage	Row before and after an update
delete	Row removed

EXAM TIP Use change data feed to propagate only changed Silver rows into Gold instead of rereading the whole table.

Upserting with Delta MERGE in PySpark

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "silver.customers")
updates = spark.read.table("bronze.customers_increment")

(target.alias("t")
  .merge(updates.alias("s"), "t.CustomerID = s.CustomerID")
  .whenMatchedUpdateAll()
  .whenNotMatchedInsertAll()
  .whenNotMatchedBySourceDelete()  # optional
  .execute())
```

Matched on the key: update the existing row

Not matched: insert as a new row

Not matched by source: target row missing from a full snapshot, so delete

Conditions: whenMatchedUpdate(condition = "s.ModifiedDate > t.ModifiedDate", set = {...})

```
MERGE INTO silver.customers t USING updates s ON t.CustomerID = s.CustomerID
WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *    (Spark SQL equivalent)
```

EXAM TRAP MERGE fails if several source rows match one target row. Deduplicate the source first.

Incremental patterns in T SQL

MERGE statement

```
MERGE dbo.DimProduct AS t
USING stg.Product AS s
  ON t.ProductID = s.ProductID
WHEN MATCHED AND t.Price <> s.Price
  THEN UPDATE SET t.Price = s.Price
WHEN NOT MATCHED BY TARGET
  THEN INSERT (ProductID, Name, Price)
  VALUES (s.ProductID, s.Name, s.Price);
```

Classic two step alternative

```
UPDATE t SET t.Price = s.Price
FROM dbo.DimProduct t
JOIN stg.Product s ON t.ProductID = s.ProductID;

INSERT INTO dbo.DimProduct
SELECT s.* FROM stg.Product s
WHERE NOT EXISTS (SELECT 1 FROM dbo.DimProduct t
  WHERE t.ProductID = s.ProductID);
```

REMEMBER Warehouse statements are ACID and can span several tables inside BEGIN TRAN and COMMIT.

EXAM TIP Land changes in a staging table first, then MERGE or UPDATE plus INSERT into the target in one transaction.

The star schema



REMEMBER Facts hold numeric measures at a declared grain plus foreign keys. Dimensions hold descriptive attributes. One to many from each dimension to the fact.

Preparing data for a dimensional model

Surrogate keys

Meaningless integer keys owned by the warehouse. They decouple from source keys and allow SCD Type 2 history.

Unknown member

A row such as key 0 or 1 labelled “Unknown”, so facts with missing references still join.

Declare the grain

One row per order line, per day per store, and so on. Every measure must match it.

Conformed dimensions

One shared DimDate or DimCustomer reused by every fact table.

```
from pyspark.sql.functions import row_number, lit
from pyspark.sql.window import Window
max_key = spark.sql("SELECT COALESCE(MAX(CustomerKey), 0) FROM gold.DimCustomer").first()[0]
new_dim = new_rows.withColumn("CustomerKey", row_number().over(Window.orderBy("CustomerID")) + lit(max_key))
```

EXAM TIP In the Warehouse, IDENTITY columns can generate surrogate keys; check current support on Microsoft Learn before relying on them.

Slowly changing dimension types

Type	Name	Behaviour when an attribute changes	History kept
0	Retain original	Ignore the change; keep the first value	None (frozen)
1	Overwrite	Update the row in place	None
2	Add new row	Expire the current row, insert a new version with a new surrogate key	Full history
3	Add new column	Move old value to a PreviousCity column	One prior value
6	Hybrid 1 + 2 + 3	New row plus a current value column on all versions	Full plus current

EXAM TIP “Report sales by the customer’s city at the time of sale” requires Type 2.

EXAM TRAP Type 1 is right for correcting typos, where history has no business value.

Type 2 in action: Ana moves from Leeds to York

Before the change

CustomerKey	CustomerID	City	ValidFrom	ValidTo	IsCurrent
101	C42	Leeds	01/03/2024	31/12/9999	1



After the load on 15/09/2026

CustomerKey	CustomerID	City	ValidFrom	ValidTo	IsCurrent
101	C42	Leeds	01/03/2024	14/09/2026	0 (expired)
187	C42	York	15/09/2026	31/12/9999	1 (current)

Old sales keep key **101** and stay in Leeds.

New sales look up the current row and get key **187**.

The business key **C42** appears on both rows.

Implementing Type 2 with one Delta MERGE

```
cur = spark.table("gold.DimCustomer").where("IsCurrent = 1")
changed = (src.alias("s").join(cur.alias("d"), "CustomerID")
  .where("s.City <> d.City").select("s.*"))

staged = (changed.withColumn("mergeKey", lit(None))
  .unionByName(src.withColumn("mergeKey", col("CustomerID"))))

(DeltaTable.forName(spark, "gold.DimCustomer").alias("t")
  .merge(staged.alias("s"), "t.CustomerID = s.mergeKey AND t.IsCurrent = 1")
  .whenMatchedUpdate(condition = "t.City <> s.City",
    set = {"IsCurrent": "0", "ValidTo": "current_date()"})
  .whenNotMatchedInsert(values = {"CustomerID": "s.CustomerID",
    "City": "s.City", "ValidFrom": "current_date()", "IsCurrent": "1"})
  .execute())
```

Rows with mergeKey match the current version, which gets expired.

Rows with null mergeKey never match, so the new version is inserted.

Brand new customers do not match either, and are inserted.

REMEMBER Expire and insert in one atomic commit.

Loading a fact table: the key lookup

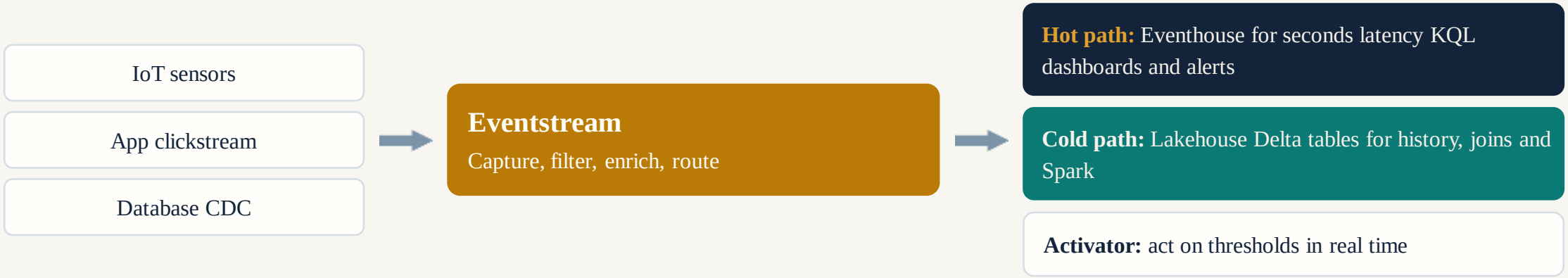
Silver orders with business keys → Join each dimension on business key and date range → Replace misses with the unknown key → Append to FactSales at the declared grain

```
d = spark.table("gold.DimCustomer")
fact = (orders.join(d, (orders.CustomerID == d.CustomerID) &
    (orders.OrderDate >= d.ValidFrom) & (orders.OrderDate <= d.ValidTo), "left")
  .withColumn("CustomerKey", coalesce(col("CustomerKey"), lit(0)))
  .select("DateKey", "CustomerKey", "ProductKey", "Quantity", "SalesAmount"))
fact.write.mode("append").saveAsTable("gold.FactSales")
```

EXAM TIP Use a left join to the dimension. An inner join silently drops facts whose dimension row has not arrived.

REMEMBER Load dimensions before facts, so the lookups find the newest keys.

Designing a loading pattern for streaming data



Design decision	Considerations
Latency target	Seconds: Eventhouse. Minutes: Spark structured streaming micro batches to Delta.
Delivery guarantee	Checkpoints for exactly once into Delta; deduplicate on event id otherwise.
Late and out of order events	Event time windows plus watermarks; tolerance agreed with the business.

CHECKPOINT 5 · LOADING PATTERNS

Test yourself: loading scenarios

Q1. A 2 billion row source table has a reliable LastModified column, but rows are sometimes hard deleted.

Answer: Watermark for inserts and updates is not enough; use CDC (or mirroring) to capture deletes.

Q2. Sales must be reported by the product category valid on the sale date.

Answer: Type 2 DimProduct with ValidFrom, ValidTo and IsCurrent, and date range key lookup.

Q3. A MERGE fails with “multiple source rows matched”. Fix?

Answer: Deduplicate the source to one row per key (latest by timestamp) before merging.

Q4. Telemetry needs live dashboards and also years of history for Spark analysis.

Answer: Eventstream to an Eventhouse (hot) and a lakehouse (cold), or Eventhouse with OneLake availability.

Lakehouse, Warehouse, Eventhouse or SQL database?

Criterion	Lakehouse	Warehouse	Eventhouse	SQL database
Data shape	Structured, semi, unstructured files	Structured	Time series, logs, events, JSON	Structured, operational
Write with	Spark, pipelines, dataflows	T SQL, pipelines, dataflows	Eventstream, KQL ingestion	Applications, T SQL
Read with	Spark, SQL endpoint (read only)	T SQL	KQL, T SQL subset	T SQL
Transactions	Single table (Delta)	Multi table ACID	Append oriented	Full OLTP
Latency	Batch, micro batch	Batch	Seconds	Milliseconds
Persona	Data engineer, scientist	SQL developer	Real time analyst	App developer

REMEMBER All four store data in OneLake, so each can be reached from the others through shortcuts or cross database queries.

Keyword to store: the exam shortcut

Lakehouse

“Images, PDFs, JSON files”, “PySpark team”, “data science”, “schema on read”, “medallion Bronze”

Warehouse

“T SQL skills”, “stored procedures”, “multi table transactions”, “UPDATE and DELETE in SQL”

Eventhouse

“Telemetry”, “logs”, “near real time”, “time series”, “KQL”, “high ingestion rate”

SQL database

“Application backend”, “OLTP”, “frequent single row updates”, “operational data store”

EXAM TIP A mixed team often lands raw data in a lakehouse and serves Gold from a Warehouse.

EXAM TRAP A requirement to write with T SQL rules out the lakehouse SQL endpoint, which is read only.

Dataflows Gen2, notebooks, KQL or T SQL?

Tool	Runs on	Strengths	Pick it when
Dataflow Gen2	Power Query mashup engine	Visual, 300 plus transforms, many connectors	Low code teams, modest volumes
Notebook	Spark cluster	Scale, Python libraries, Delta MERGE, ML	Big data, complex logic, files
KQL	Eventhouse engine	Fast time series, parsing, update policies	Telemetry already in an Eventhouse
T SQL	Warehouse engine	Set based SQL, procedures, transactions	SQL team, data in a Warehouse

Data in a lakehouse
Notebook first

Data in a Warehouse
T SQL first

Data in an Eventhouse
KQL first

No code requirement
Dataflow Gen2

> **EXAM TIP** Transform where the data already lives. Moving data just to use a favourite tool costs time and capacity.

Shortcuts: data virtualisation without copies

Sales_Gold lakehouse

Tables/

fact_sales (native Delta)

dim_product (shortcut)

Files/

partner_feeds (shortcut)

Internal target: Product_Master lakehouse in another workspace (OneLake to OneLake)

External target: Amazon S3 bucket owned by a partner

Pointer only: no data is copied; queries read the source live.

Transparent: Spark, SQL and Power BI see it as a normal table or folder.

Safe delete: removing a shortcut never deletes the target data.

Shortcut sources and placement rules

Supported targets

OneLake (internal) | ADLS Gen2 | Azure Blob Storage | Amazon S3 | S3 compatible | Google Cloud Storage | Dataverse | On premises via gateway | SharePoint and OneDrive

Shortcuts can be created in lakehouses, KQL databases and other OneLake items.

Where a shortcut can live

Area	Rule
Tables	Top level only; target must be a Delta table folder to appear as a table
Files	Any folder level; any file format
Nesting	Shortcuts can point to shortcuts, up to a chain limit

EXAM TRAP A shortcut to Parquet or CSV files placed under Tables shows as “Unidentified”. Put non Delta data under Files, or convert it to Delta.

Shortcut security: whose identity is used?

Internal (OneLake) shortcuts

Passthrough: the calling user's identity is checked against the **target** item. The user needs access to the target data.

User A reads shortcut, OneLake checks User A on the target lakehouse

External shortcuts

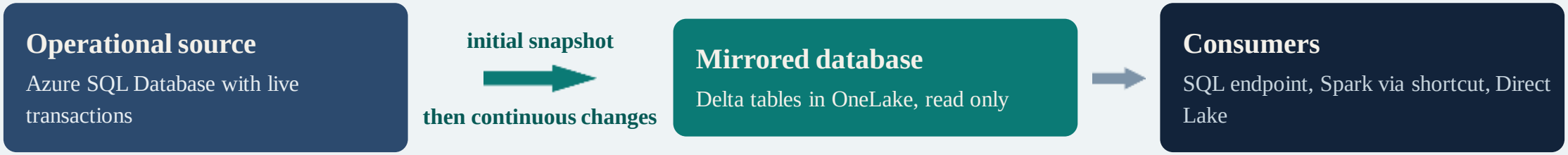
Delegated: a stored **connection** credential (account key, SAS, service principal, workspace identity) reaches the source. Fabric permissions then control who reads the shortcut.

User A reads shortcut, connection credential reads S3

REMEMBER Creating a shortcut needs Write on the item where it is created, and Read on the target.

EXAM TIP Use workspace identity for ADLS Gen2 behind a firewall with trusted workspace access.

Mirroring: a near real time replica in OneLake



No pipelines. Fabric manages replication, schema and change tracking.

Low cost. Replication compute is free, plus free mirror storage up to a limit tied to the capacity size.

Read only. Transform downstream with notebooks, views or a Warehouse.

EXAM TIP “Analyse operational data with minimal latency and no ETL development” is mirroring.

Mirroring types and sources

Type	Sources (examples)	How it works
Database mirroring	Azure SQL Database, Azure SQL Managed Instance, SQL Server, Azure Cosmos DB, Azure Database for PostgreSQL, Snowflake	Full replica of selected tables, kept in sync from the source change log
Metadata mirroring	Azure Databricks Unity Catalog	Shortcuts to existing Delta tables; catalog structure only
Open mirroring	Any application or partner tool	Writer drops Parquet change files into a landing zone; Fabric applies them

Setup: create a Mirrored database item, choose a connection, select tables or mirror all, start mirroring.

Monitor: replication status, rows replicated and last completed time per table.

EXAM TRAP Not every data type or table is supported. Tables without a primary key, or with unsupported column types, may be skipped or need changes.

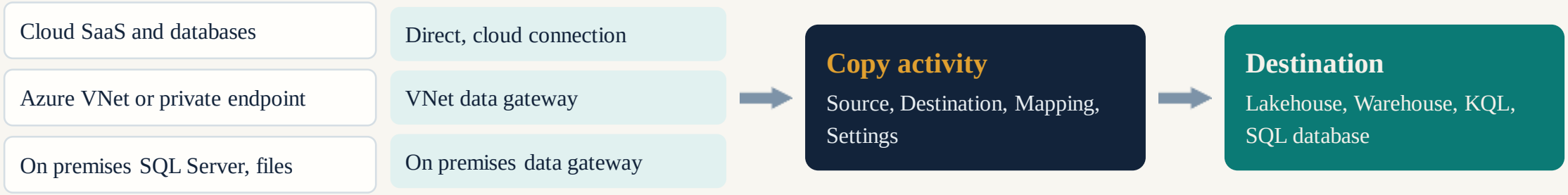
Shortcut, mirroring, Copy job or pipeline?

Need	Shortcut	Mirroring	Copy job	Pipeline Copy
Data copied?	No	Yes, replica	Yes	Yes
Freshness	Live at query	Near real time	Scheduled, incremental	Scheduled or triggered
Sources	Storage and OneLake	Supported databases	Many connectors	Widest connector set
Effort	Lowest	Low	Low, wizard driven	Medium to high
Control flow	None	None	Minimal	Full
Transform	No	No	Column mapping	Mapping, then other activities

EXAM TIP “Avoid data duplication” points to a shortcut. “Database source, continuous sync” points to mirroring.

REMEMBER Copy job offers full, incremental and CDC style copies without building a pipeline.

The Copy activity and how it reaches sources



Tab	Key options
Source	Table, query or file path with wildcards; dynamic content for parameters
Destination	Table action Append or Overwrite; file format; partitioning
Mapping	Import schemas, rename and retype columns
Settings	Throughput optimisation, parallel copies, staging, fault tolerance, logging

EXAM TIP Data behind a corporate firewall on premises always needs an on premises data gateway.

Copy activity settings that matter

Intelligent throughput optimisation

Auto, Standard, Balanced or Maximum compute for the copy. Higher settings move data faster and consume more CUs.

Degree of copy parallelism

Parallel threads reading partitions or files. Pair with source partition options (physical or dynamic range).

Staging

Land data in interim storage first. Needed for some sources and for efficient bulk loads into a Warehouse.

Fault tolerance and logging

Skip incompatible rows instead of failing; log skipped rows to storage for review.

EXAM TIP “A few bad rows must not stop the whole load, but must be reviewable” is fault tolerance with logging.

REMEMBER Copy into a lakehouse table writes Delta; choose Append for incremental and Overwrite for full loads.

Copy job: simple movement as an item

A **Copy job** is a dedicated item that moves data from a source to a destination with a guided wizard, its own schedule and built in incremental logic, without designing a pipeline.

Full copy · every run copies all rows

Incremental copy · first run full, then only new or changed rows by an incremental column or CDC

Write behaviour · append or merge (upsert) on keys

Many tables at once in one job, each with its own settings.

State kept for you: the job remembers its own watermark.

Callable from a pipeline with the Copy job activity.

EXAM TIP Pick Copy job for “incremental copy with the least development effort”.

PySpark foundations: reading and writing

Read

```
df = (spark.read.format("csv")
  .option("header", "true")
  .option("inferSchema", "true")
  .load("Files/raw/orders/*.csv"))

js = spark.read.json("Files/raw/events/")
t = spark.read.table("silver.customers")
q = spark.sql("SELECT * FROM silver.customers")
```

Write

```
(df.write.format("delta")
  .mode("overwrite")  # append, overwrite,
          # error, ignore
  .partitionBy("OrderYear")
  .option("overwriteSchema", "true")
  .saveAsTable("silver.orders"))

df.write.mode("append").save("Tables/orders")
```

EXAM TIP Relative paths Files/ and Tables/ resolve against the notebook's default lakehouse.

EXAM TRAP inferSchema scans data and can mistype columns. Define a schema with StructType for production loads.

The transformations you will see most

```
from pyspark.sql.functions import col, when, to_date, upper, lit, year

clean = (df
  .select("OrderID", "CustomerID", "Amount", "OrderDate", "Status")
  .filter(col("Amount") > 0)
  .withColumn("OrderDate", to_date("OrderDate", "dd/MM/yyyy"))
  .withColumn("OrderYear", year("OrderDate"))
  .withColumn("Status", upper(col("Status")))
  .withColumn("Band", when(col("Amount") >= 1000, "High")
    .otherwise("Standard"))
  .withColumnRenamed("CustomerID", "CustKey")
  .drop("Status"))
```

- **select** chooses columns
- **filter / where** keeps rows
- **withColumn** adds or replaces
- **when / otherwise** is CASE
- **cast** changes data type
- **withColumnRenamed**, **drop**

REMEMBER Transformations are lazy. Nothing runs until an action such as write, count, show or collect.

Spark SQL and lakehouse table types

```
%%sql
CREATE TABLE IF NOT EXISTS silver.orders (
  OrderID INT, CustomerID INT,
  Amount DECIMAL(18,2), OrderDate DATE
) USING DELTA
PARTITIONED BY (OrderDate);

INSERT INTO silver.orders
SELECT OrderID, CustomerID, Amount, OrderDate
FROM bronze.orders_raw WHERE Amount IS NOT NULL;

CREATE OR REPLACE TEMP VIEW recent AS
SELECT * FROM silver.orders WHERE OrderDate >= date_sub(current_date(), 30);
```

Managed table
Spark owns data and metadata in Tables/. DROP TABLE deletes the files.

External table
Created with a LOCATION. DROP TABLE removes metadata only; files remain.

REMEMBER %%sql switches a cell to Spark SQL; spark.sql() runs the same from Python.

Loading and shaping data in the Warehouse

COPY INTO: bulk load files

```
COPY INTO dbo.Sales
FROM 'https://acct.blob.core.windows.net/raw/sales/*.parquet'
WITH (FILE_TYPE = 'PARQUET');
```

CTAS: create from a query

```
CREATE TABLE gold.SalesByRegion AS
SELECT Region, SUM(Amount) AS Total
FROM dbo.Sales GROUP BY Region;
```

INSERT ... SELECT across items

```
INSERT INTO dbo.DimCustomer (CustomerID, Name)
SELECT CustomerID, Name
FROM SalesLakehouse.dbo.customers;
```

OPENROWSET: query files ad hoc

```
SELECT TOP 10 *
FROM OPENROWSET(BULK 'https://acct.dfs.core.windows.net/raw/x.parquet')
AS r;
```

EXAM TIP Three part names (Item.schema.table) join a Warehouse with lakehouse SQL endpoints in the same workspace, with no data movement.

Procedures, views and transactions

```
CREATE PROCEDURE etl.usp_LoadFactSales @LoadDate DATE
AS
BEGIN
  BEGIN TRANSACTION;
  DELETE FROM gold.FactSales WHERE OrderDate = @LoadDate;
  INSERT INTO gold.FactSales
  SELECT d.DateKey, c.CustomerKey, s.Amount
  FROM stg.Sales s
  JOIN gold.DimDate d ON d.Date = s.OrderDate
  LEFT JOIN gold.DimCustomer c
    ON c.CustomerID = s.CustomerID AND c.IsCurrent = 1
  WHERE s.OrderDate = @LoadDate;
  COMMIT TRANSACTION;
END;
```

Delete then insert for a date makes reruns idempotent.

Transaction keeps the fact table consistent if anything fails.

Call it from a pipeline Stored procedure activity with a parameter.

EXAM TRAP Views and procedures can be created on a SQL endpoint, but INSERT into lakehouse tables cannot.

KQL in one slide: the pipe and its operators

Telemetry | where | extend | summarize | order by | project Data flows left to right through each operator

Operator	Does	SQL equivalent
where	Filter rows	WHERE
project / project away	Choose or drop columns	SELECT list
extend	Add calculated columns	SELECT expression AS name
summarize ... by	Aggregate by groups	GROUP BY
join kind=inner / leftouter	Combine tables	JOIN (default KQL kind is innerunique)
take / top / order by	Sample, rank, sort	TOP, ORDER BY
distinct / count	Unique values, row count	DISTINCT, COUNT(*)
parse / mvexpand	Split strings; expand arrays into rows	String functions; CROSS APPLY

A worked KQL query, annotated

```
DeviceTelemetry
| where Timestamp > ago(1h)
| where DeviceType == "Freezer"
| extend TempF = TempC * 9.0 / 5 + 32
| summarize AvgTemp = avg(TempC),
    MaxTemp = max(TempC), Readings = count()
  by DeviceId, bin(Timestamp, 5m)
| where MaxTemp > 0
| order by MaxTemp desc
| project DeviceId, Timestamp, AvgTemp, MaxTemp, Readings
```

ago(1h): time relative to now; filter time first

== is case sensitive; =~ ignores case

extend keeps every column and adds TempF

bin(Timestamp, 5m): 5 minute buckets

where after summarize acts like HAVING

EXAM TIP Filter early on time and use has rather than contains for speed.

Denormalisation: fewer joins at query time

Normalised Silver (snowflake)



Three tables, two joins for every report

Denormalised Gold dimension

Product	Subcategory	Category
Road Bike 52	Road	Bikes
Helmet Pro	Helmets	Accessories

One flat table, attributes repeated

Gain: simpler star schema, faster Direct Lake and SQL queries.

Cost: more storage and wider tables; updates touch more rows.

Also: flatten nested JSON into columns and arrays into rows.

Denormalising in code

Join and flatten (PySpark)

```
from pyspark.sql.functions import broadcast
dim_product = (product
  .join(broadcast(subcat), "SubcategoryID", "left")
  .join(broadcast(cat), "CategoryID", "left")
  .select("ProductID", "ProductName",
    "SubcategoryName", "CategoryName"))
```

Nested JSON to rows (PySpark)

```
from pyspark.sql.functions import explode, col
lines = (orders
  .select("OrderID", explode("items").alias("item"))
  .select("OrderID", col("item.sku"), col("item.qty")))
```

Same idea in T SQL and KQL

```
SELECT p.ProductID, p.Name, s.Name AS Subcategory,
  c.Name AS Category
FROM Product p
LEFT JOIN Subcategory s ON s.ID = p.SubcategoryID
LEFT JOIN Category c ON c.ID = s.CategoryID;
```

```
Orders
| mvexpand item = Items
| extend Sku = tostring(item.sku), Qty = toint(item.qty)
| lookup kind=leftouter Products on Sku
```

EXAM TIP broadcast() a small dimension to avoid shuffling the large table.

One aggregation, three languages

Goal: total sales, order count and average order value per region and year.

PySpark

```
from pyspark.sql import functions as F
(df.groupBy("Region", "Year")
 .agg(F.sum("Amount").alias("Total"),
   F.countDistinct("OrderID").alias("Orders"),
   F.avg("Amount").alias("AOV"))
 .orderBy(F.desc("Total")))
```

T SQL or Spark SQL

```
SELECT Region, Year,
  SUM(Amount) AS Total,
  COUNT(DISTINCT OrderID) AS Orders,
  AVG(Amount) AS AOV
FROM sales
GROUP BY Region, Year
HAVING SUM(Amount) > 10000
ORDER BY Total DESC;
```

KQL

```
Sales
| summarize Total = sum(Amount),
  Orders = dcount(OrderID),
  AOV = avg(Amount)
  by Region, Year
| where Total > 10000
| order by Total desc
```

EXAM TRAP KQL dcount() is approximate by design. Use count_distinct() when an exact distinct count is required.

Window functions: aggregates that keep every row

Region	Sales	ROW_NUMBER	RANK	DENSE_RANK
North	900	1	1	1
North	700	2	2	2
North	700	3	2	2
North	500	4	4	3

Ties: ROW_NUMBER breaks them, RANK leaves a gap, DENSE_RANK does not.

```
SELECT Region, Sales,
  RANK() OVER (PARTITION BY Region ORDER BY Sales DESC) AS Rnk,
  SUM(Sales) OVER (PARTITION BY Region) AS RegionTotal,
  LAG(Sales) OVER (PARTITION BY Region ORDER BY Month) AS Prev
FROM monthly_sales;
```

```
w = Window.partitionBy("Region").orderBy(F.desc("Sales"))
df.withColumn("Rnk", F.rank().over(w))
  .withColumn("Running", F.sum("Sales").over(
    w.rowsBetween(Window.unboundedPreceding, 0)))
```

REMEMBER PARTITION BY defines the group, ORDER BY the sequence, and the frame (ROWS BETWEEN) the running window.

Removing duplicates the right way

Exact duplicates (PySpark)

```
df.dropDuplicates()  # whole row
df.dropDuplicates(["OrderID"])  # by key, arbitrary row kept
```

Keep the latest per key (PySpark)

```
w = Window.partitionBy("OrderID").orderBy(F.desc("UpdatedAt"))
latest = (df.withColumn("rn", F.row_number().over(w))
  .filter("rn = 1").drop("rn"))
```

T SQL

```
WITH r AS (SELECT *, ROW_NUMBER() OVER
  (PARTITION BY OrderID ORDER BY UpdatedAt DESC) AS rn
  FROM stg.Orders)
SELECT * FROM r WHERE rn = 1;
```

KQL

```
Orders
| summarize arg_max(UpdatedAt, *) by OrderID

Orders | distinct OrderID, Status
```

EXAM TIP dropDuplicates on a key keeps an unpredictable row. When “the most recent version” matters, use ROW_NUMBER or arg_max.

Strategies for missing values

Drop
Remove rows missing a mandatory field such as the key.

Default
Replace with “Unknown”, 0 or a sentinel date.

Impute
Mean, median or last known value.

Quarantine
Route bad rows to an error table for review.

```
df.na.drop(subset=["OrderID"])
df.na.fill({"Region": "Unknown", "Qty": 0})
df.withColumn("City", F.coalesce("City", F.lit("N/A")))
bad = df.filter(F.col("CustomerID").isNull())
```

```
SELECT COALESCE(Region, 'Unknown') AS Region,
  ISNULL(Qty, 0) AS Qty FROM stg.Sales
WHERE OrderID IS NOT NULL;

T | extend Region = iff(isempty(Region), "Unknown", Region)
```

EXAM TRAP Replacing a missing numeric measure with 0 changes averages. Prefer NULL or imputation when the value is truly unknown.

Late arriving facts and early arriving facts

Late arriving fact

An event from last week arrives today. Load it by **event date**, not load date, and look up the dimension version valid at the event date.

Reprocess affected partitions or aggregates idempotently.

Early arriving fact (late dimension)

A sale references customer C99 who is not yet in DimCustomer. Insert an **inferred member** row with the business key and placeholder attributes.

Update the inferred row in place (Type 1) when real data arrives.

In streaming Event time → Watermark: accept up to 10 minutes late → Later events dropped or sent to a side output

EXAM TIP “Facts must not be lost when the dimension row is missing” means inferred members or the unknown member, never an inner join.

Dataflow Gen2: Power Query at Fabric scale

Get data
Hundreds of connectors, gateways for on premises

Transform
Applied steps recorded as M code

Destination
Lakehouse, Warehouse, KQL, SQL database, files

Publish and refresh
Save, then schedule or call from a pipeline

```
let
  Source = Lakehouse.Contents(null),
  Orders = Source{[Id = "orders"]}[Data],
  Typed = Table.TransformColumnTypes(Orders,
    {{"Amount", type number}}),
  Kept = Table.SelectRows(Typed, each [Amount] > 0)
in
  Kept
```

Query folding pushes steps back to the source, which is fastest. Watch the folding indicators.

Public parameters let a pipeline pass values at refresh.

CI/CD support stores the definition in Git.

Dataflow Gen2 settings examiners love

Update method

Replace drops and reloads the destination; **Append** adds rows. Choose per query.

Staging

Lands data in a staging lakehouse and uses Warehouse compute for heavy transforms on large data. Disable it for small queries to save time.

Fast copy

Uses the pipeline copy engine for large volumes when transforms are simple, such as column selection and type changes.

Incremental refresh

Refreshes only recent buckets based on a DateTime column, instead of the whole table.

EXAM TIP A slow Dataflow Gen2 on millions of rows: check query folding, enable fast copy, or move the logic to a notebook.

EXAM TRAP Replace on a destination table wipes rows loaded by other processes.

CHECKPOINT 6 · BATCH DATA

Test yourself: batch scenarios

Q1. Finance wants partner data in S3 available in the lakehouse without copying it.

Answer: An Amazon S3 shortcut, placed under Files (or Tables if the data is Delta).

Q2. Keep the newest record per CustomerID in KQL. Complete: summarize ____(UpdatedAt, *) by CustomerID

Answer: arg_max

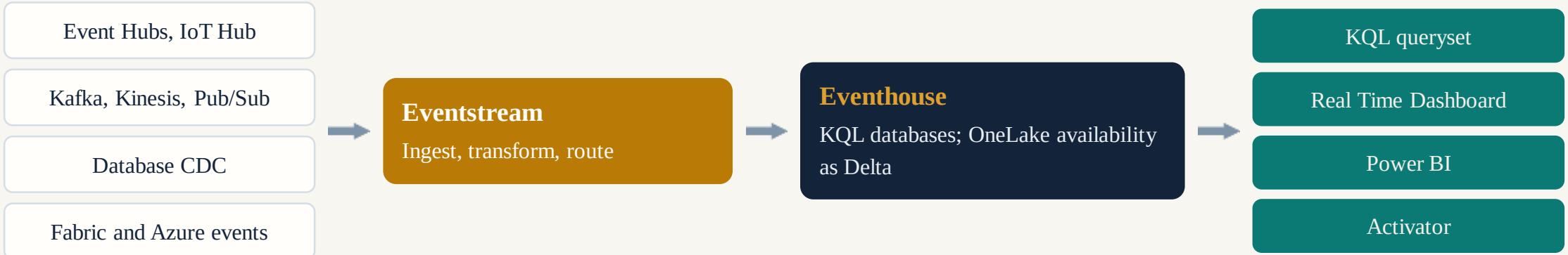
Q3. An Azure SQL database must be analysed in Fabric within seconds of changes, no ETL.

Answer: Database mirroring into a Mirrored database item.

Q4. Join a 2 TB fact to a 5 MB lookup table efficiently in PySpark.

Answer: Broadcast the small table: fact.join(broadcast(lookup), "key").

Real Time Intelligence end to end



Real Time hub: tenant wide catalogue of streams and events to discover and connect.

Activator: no code rules that send alerts or start pipelines when conditions are met.

Spark streaming is the code first alternative landing Delta tables.

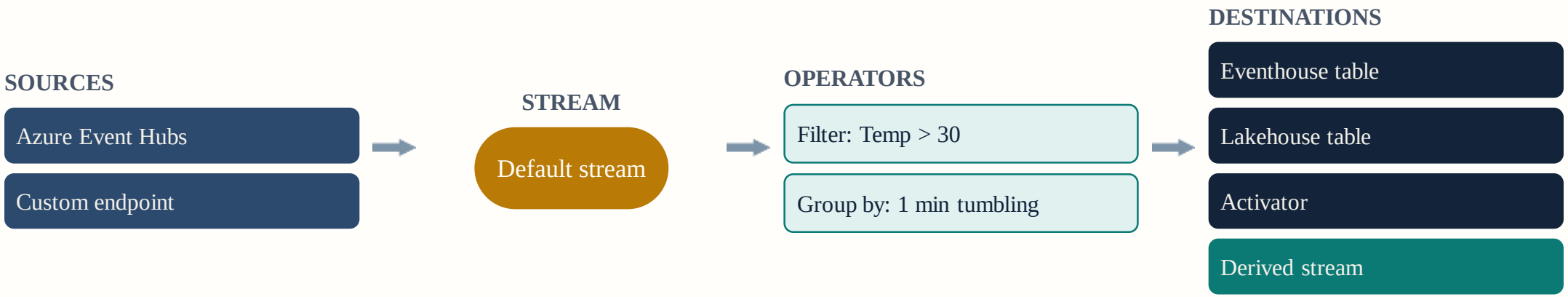
Eventstream, Spark structured streaming or KQL?

Criterion	Eventstream	Spark structured streaming	KQL (Eventhouse)
Style	No code designer	Code in notebooks or Spark job definitions	Query language, policies
Latency	Seconds	Seconds to minutes (micro batch)	Seconds after ingestion
Transformations	Filter, fields, aggregate, join, union	Anything Spark can do, libraries, ML	Update policies, materialized views
Output	Eventhouse, lakehouse, Activator, endpoints	Delta tables in a lakehouse	KQL tables, dashboards
Best for	Routing and light shaping	Complex logic into Delta	Time series analytics at scale

EXAM TIP “No code” plus “real time” is Eventstream. “Complex Python logic on streams into Delta” is Spark.

REMEMBER Most designs combine them: Eventstream ingests, KQL analyses, Spark enriches history.

Structure of an Eventstream



Edit mode changes nothing until you publish; live view shows data flowing.

Derived stream publishes a transformed stream to the Real Time hub for reuse.

Fan out: one source can feed many destinations in parallel.

Eventstream sources and destinations

Sources

- Azure Event Hubs, Azure IoT Hub
- Apache Kafka, Confluent Cloud, Amazon MSK
- Amazon Kinesis, Google Cloud Pub/Sub
- CDC: Azure SQL, PostgreSQL, MySQL, Cosmos DB
- Custom endpoint (Event Hub, AMQP, Kafka protocols)
- Fabric workspace, OneLake and job events
- Azure Blob Storage events, sample data

Destinations

- Eventhouse: direct ingestion, or processed before ingestion
- Lakehouse Delta table
- Activator for alerts and actions
- Custom endpoint for downstream apps
- Derived stream

REMEMBER Direct ingestion lets the Eventhouse pull raw events; processed mode applies operators first.

EXAM TIP An application that cannot use Azure SDKs can still send events to a custom endpoint through the Kafka protocol.

Eventstream operators

Filter
Keep events meeting a condition

Manage fields
Add, remove, rename, change type, built in functions

Aggregate
Running sum, min, max, average over a period

Group by
Aggregates per key inside a time window

Join
Combine two streams on a key and time window

Union
Merge streams with matching schemas

Expand
Turn an array field into one row per element

SQL code
Express logic as a SQL like stream query

Content routing
Send different events to different destinations

EXAM TRAP Joining a stream to a large reference table is not Eventstream’s strength. Ingest first, then enrich in KQL with lookup or an update policy.

Inside an Eventhouse

Eventhouse (shared compute and cache)

KQL database: Telemetry
Tables, functions, materialized views, shortcuts

KQL database: Logs
Separate policies and permissions

Data is stored as compressed, indexed columnar extents; hot data stays in the SSD cache.

Indexes everything, including free text and dynamic JSON columns.

OneLake availability exposes tables as Delta for Spark and Direct Lake, without a second copy.

Minimum consumption keeps compute always available for predictable latency.

Query with KQL querysets, dashboards, or a T SQL subset.

REMEMBER Management commands start with a dot, such as .create table, .alter policy, .show ingestion failures.

Getting data into a KQL table

Queued (batched) ingestion

Events are collected and sealed into extents by the **ingestion batching policy**: whichever comes first of time, item count or size.

Efficient, high throughput; small delay

Streaming ingestion

Rows become queryable within seconds. Enabled by the **streaming ingestion policy** on the table or database.

Lowest latency; for small, frequent payloads

```
.create table Telemetry (DeviceId: string, Timestamp: datetime, TempC: real, Payload: dynamic)
.create table Telemetry ingestion json mapping "TelemetryMap"
 '[{"column":"DeviceId","path":"$.device"},{"column":"TempC","path":"$.temp"}]'
.alter table Telemetry policy streamingingestion enable
```

EXAM TIP An ingestion mapping translates JSON paths or CSV ordinals into table columns. Missing or wrong mappings cause empty columns.

Native tables or OneLake shortcuts?

Native KQL table

Data is ingested into the Eventhouse.

- Fastest queries, full indexing
- Update policies, materialized views
- Retention and caching policies
- Streaming ingestion supported

```
Telemetry | where Timestamp > ago(1h)
```

OneLake shortcut

Delta data stays in the lakehouse; KQL reads it as an external table.

- No duplication, no ingestion cost
- Join live telemetry with lakehouse dimensions
- Slower than native, fewer features
- Read only from KQL

```
external_table("DimDevice")
```

EXAM TIP High frequency, low latency queries on streaming data: native tables. Occasional joins to reference data already in a lakehouse: shortcut.

Query acceleration versus standard shortcuts

Relative query speed (illustrative)



Longer bar means faster. Shape only, not measured figures.

Query acceleration indexes and caches the shortcut's Delta data inside the Eventhouse, for near native speed.

You set a caching period, so only recent data is accelerated.

Standard shortcuts read Delta files at query time: no extra cost, slower.

Trade off: acceleration consumes extra capacity and cache storage.

EXAM TIP "Frequent dashboard queries over lakehouse data from KQL, without ingesting it" points to an accelerated shortcut.

Structured streaming: an unbounded table

readStream
Event Hubs, Kafka, Delta table, files

→

Transform
Same DataFrame API as batch

→

writeStream
Delta table with a checkpoint

Output modes

append: new rows only
complete: whole result each time
update: changed rows only

Triggers

processingTime: every N seconds
availableNow: process backlog, then stop
default: next batch when ready

Checkpoint

Stores offsets and state, so a restarted query resumes exactly where it stopped. One per query.

EXAM TIP Run long lived streams as a Spark job definition with a retry policy, not in an interactive notebook session that times out.

A complete streaming job, annotated

```
from pyspark.sql.functions import window, avg, col

raw = (spark.readStream.format("delta")
  .table("bronze.sensor_events"))

agg = (raw
  .withWatermark("EventTime", "10 minutes")
  .groupBy(window(col("EventTime"), "5 minutes"), "DeviceId")
  .agg(avg("TempC").alias("AvgTemp")))

(agg.writeStream.format("delta")
  .outputMode("append")
  .option("checkpointLocation", "Files/checkpoints/sensor_agg")
  .trigger(processingTime = "1 minute")
  .toTable("silver.sensor_5min"))
```

readStream on Delta picks up new commits incrementally.

withWatermark bounds state and drops events over 10 minutes late.

window() creates 5 minute tumbling buckets.

append with a watermark emits each window once it closes.

EXAM TRAP Aggregation in append mode without a watermark fails.

Update policies and materialized views



Update policy

A function runs on each new batch in the source table and writes its output to a target table. Ideal for parsing, filtering and enrichment.

Materialized view

A summarize query kept up to date in the background. Queries read the precomputed part plus the newest delta.

EXAM TIP “Keep only the latest record per device” is a materialized view using arg_max; “transform on arrival” is an update policy.

The KQL commands behind the pattern

1 · Transform function

```
.create function ParseRaw() { RawEvents
  | extend DeviceId = tostring(Payload.device), TempC = toreal(Payload.temp)
  | project Timestamp, DeviceId, TempC }
```

2 · Attach update policy

```
.alter table CleanEvents policy update
  @'[{"IsEnabled": true, "Source": "RawEvents", "Query": "ParseRaw()", "IsTransactional": true}]'
```

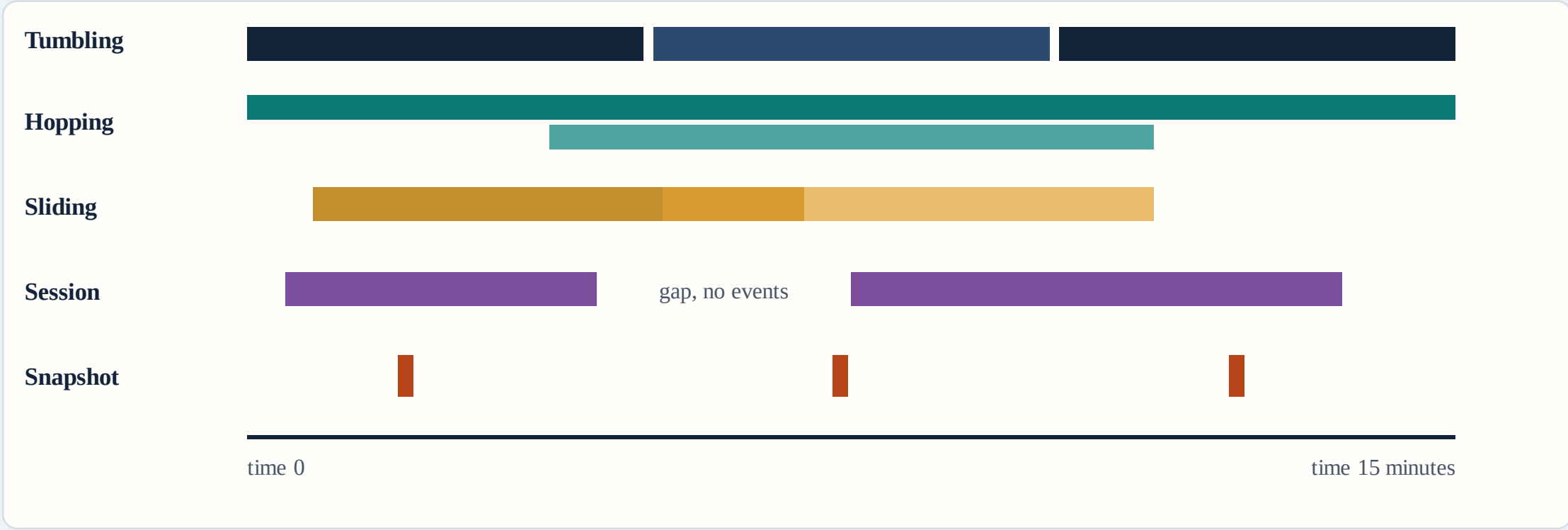
3 · Materialized view

```
.create materialized view LatestByDevice on table CleanEvents
{ CleanEvents | summarize arg_max(Timestamp, *) by DeviceId }
```

REMEMBER IsTransactional true means a failed transformation also fails ingestion into the source table.

EXAM TRAP Function output columns must match the target table schema exactly.

The five streaming windows, on one timeline



Window definitions and how to write them

Window	Behaviour	Spark	KQL
Tumbling	Fixed size, no overlap; each event in one window	window(col("ts"), "5 minutes")	summarize ... by bin(ts, 5m)
Hopping	Fixed size, overlapping by a hop; events in several windows	window(col("ts"), "10 minutes", "5 minutes")	Expand each event to several bins, or sliding_window_counts
Sliding	Output only when an event enters or leaves the window	Approximated with a short slide	Approximated by expanding events into fine bins
Session	Groups bursts; closes after a gap of inactivity	session_window(col("ts"), "5 minutes")	row_window_session(ts, 1h, 5m)
Snapshot	Groups events with the same timestamp	groupBy("ts")	summarize ... by ts

REMEMBER Eventstream Group by offers all five window types in the designer.

EXAM TIP “Every 5 minutes, report the last 10 minutes” is hopping. “Per user visit” is session.

Test yourself: streaming scenarios

Q1. Count clicks per page every minute over the previous five minutes.

Answer: Hopping window, size 5 minutes, hop 1 minute.

Q2. Raw JSON must be parsed into a typed table automatically as it arrives in the Eventhouse.

Answer: A KQL update policy on the target table using a parsing function.

Q3. A Spark stream must resume after a crash without duplicating rows in Delta.

Answer: A stable checkpointLocation for the query.

Q4. KQL dashboards must join live data to a product table in a lakehouse, fast, without ingesting it.

Answer: A OneLake shortcut with query acceleration enabled.

DOMAIN 3 · 30 TO 35% OF THE EXAM

Monitor and optimize an analytics solution

03

- Monitor Fabric items
- Identify and resolve errors
- Optimize performance

The monitoring toolkit at three altitudes

Capacity	**Fabric Capacity Metrics app:** CU usage by item and operation, overload, throttling, smoothing.
Tenant and workspace	**Monitor hub:** every run of every item you can access. **Workspace monitoring:** detailed logs in an Eventhouse you query with KQL.
Item	**Run history** for pipelines and dataflows, **Spark UI** and application monitoring, **query insights** in the Warehouse, **data insights** in Eventstream, **refresh history** for semantic models.
React	**Alerts:** Activator rules, pipeline Outlook or Teams activities, refresh failure notifications.

EXAM TIP Match the question’s scope: “which item consumes the most capacity” is the Metrics app; “why did last night’s run fail” is the Monitor hub.

The Monitor hub

Filter: Status = Failed | Item type = Pipeline | Start time: last 24 hours

Activity name	Status	Item type	Start	Duration
PL_Daily_Load	Failed	Pipeline	02:00	4 min 12 s
NB_Silver_Orders	Succeeded	Notebook	02:05	9 min 40 s
DF_Clean_Customers	In progress	Dataflow Gen2	02:16	running

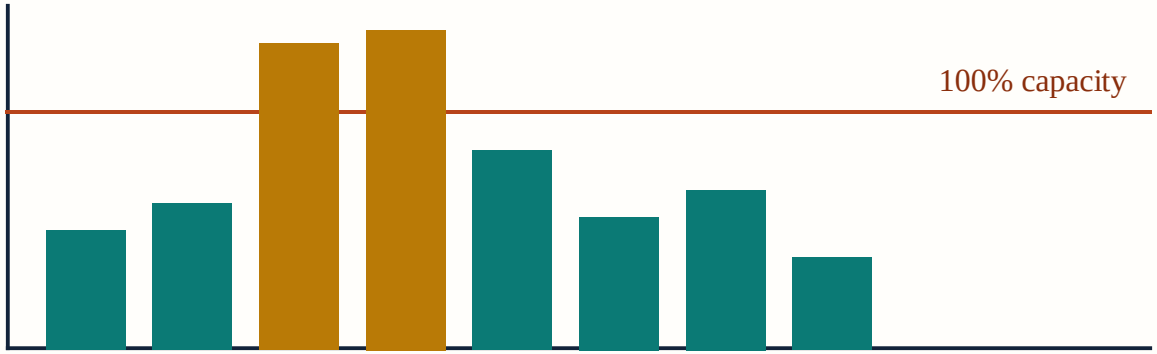
Covers pipelines, dataflows, notebooks, Spark jobs, semantic model refreshes, Copy jobs and more.

Drill in from a row to the item's detailed run view, inputs, outputs and error.

Scope: shows only items you have permission to view; history is time limited.

Reading the Capacity Metrics app

CU utilisation over a day (illustrative)



Amber bars exceed the line: overage that later triggers throttling.

Compute page: CU by item, operation, user; interactive versus background.

Throttling chart: delay and rejection percentages against limits.

Timepoint drill through: exactly which operations ran in a 30 second slot.

Storage page: OneLake storage by workspace.

REMEMBER Capacity admins install and use the app; it is the tool for chargeback and right sizing a SKU.

Workspace monitoring: logs you can query

- Semantic model queries and refreshes
- Eventhouse queries and ingestion
- Mirrored database and other item logs

Monitoring Eventhouse
Created when an admin turns on workspace monitoring

Query and alert
KQL, dashboards, Activator

```
SemanticModelLogs
| where Timestamp > ago(1d) and OperationName == "QueryEnd"
| summarize AvgMs = avg(DurationMs), Queries = count() by ItemName
| order by AvgMs desc
```

EXAM TIP “Keep detailed operational logs and query them with KQL” is workspace monitoring.

REMEMBER Enable it in Workspace settings, Monitoring; it consumes capacity and has a retention period.

Monitoring ingestion, item by item

Item	Where to look	Key signals
Pipeline	Run history, then activity runs; Copy activity details pane	Rows read and written, throughput, duration breakdown, errors
Copy job	Copy job monitoring pane and Monitor hub	Rows copied per table, incremental state
Mirroring	Mirrored database, Monitor replication	Status per table, rows replicated, last sync time
Eventstream	Data insights and runtime logs on each node	Incoming and outgoing events, errors, backlog
Eventhouse	.show ingestion failures, database overview	Failed ingestions, ingestion latency, row counts
Dataflow Gen2	Refresh history	Per query duration, rows, status

EXAM TIP A Copy activity that is slow: open its details and read the duration breakdown (queue, read, write, transfer) to find the bottleneck.

Monitoring Spark and Warehouse transformations

Spark

- Notebook cell progress and Spark job list
- Spark application detail: jobs, stages, executors, logs
- Spark UI and Spark history server
- Resource utilisation chart: executors over time
- Spark advisor recommendations and skew warnings
- Run series: compare durations across recurring runs

Warehouse and SQL endpoint

- queryinsights.exec_requests_history
- queryinsights.long_running_queries
- queryinsights.frequently_run_queries
- sys.dm_exec_requests for live sessions
- Query activity tab in the Warehouse editor

```
SELECT TOP 10 command, total_elapsed_time_ms, start_time
FROM queryinsights.exec_requests_history ORDER BY total_elapsed_time_ms DESC;
```

EXAM TIP One stage taking far longer than the others, with a few tasks much slower than the rest, signals data skew.

Semantic model refresh: import and Direct Lake

Import mode

Refresh copies data into the model. Watch duration, memory and failures in **Refresh history** and the Monitor hub.

Direct Lake mode

Refresh is **framing**: a quick metadata update pointing the model at the latest Delta version. Automatic updates can do it when data changes.

Refresh history in model settings: start, end, status, error text.

Failure notifications by email to the owner or named contacts.

Pipeline activity to refresh after the load, with its own run status.

EXAM TRAP Direct Lake can fall back to DirectQuery (on the SQL endpoint flavour) when guardrails are exceeded or views are used, which slows reports.

EXAM TIP Trigger the refresh at the end of the load pipeline so reports never show partial data.

Configuring alerts

- Eventstream or KQL data
- Real Time Dashboard tile
- Power BI visual
- Fabric job events (run failed)

Activator

Object, property, condition (rises above, changes, no data for)

- Email
- Teams message
- Run a pipeline or notebook
- Power Automate flow

Inside a pipeline: Office 365 Outlook or Teams activity on the failure path, with dynamic content such as @pipeline().RunId.

Across all items: Activator on job events alerts on any failed scheduled run without editing each item.

EXAM TIP “Alert when a freezer stays above zero for 10 minutes” is an Activator rule on streaming data, not a scheduled query.

A five step troubleshooting framework

1 — Locate
Monitor hub: which item, which run, which activity

2 — Read
Error code and message; inputs and outputs

3 — Classify
Connectivity, permission, data, code, capacity

4 — Fix
Smallest change at the root cause

5 — Prevent
Retries, validation, alerts, idempotent reruns

Error class	Typical symptoms	First check
Connectivity	Timeouts, host not found, gateway offline	Gateway status, firewall, credentials
Permission	403 Forbidden, access denied	Role, item permission, connection identity
Data	Conversion failed, schema mismatch, nulls	Source sample, mapping, data types
Capacity	Throttled, queued, rejected requests	Capacity Metrics app

Diagnosing pipeline failures

Symptom	Likely cause	Resolution
Copy fails: user configuration issue, column type mismatch	Source type cannot convert to sink type	Fix mapping, cast in source query, enable fault tolerance
Expression evaluation failed	Typo in activity name, wrong output path, null value	Check exact activity name; use coalesce; test in debug
Cannot connect to on premises source	Gateway offline or wrong credentials	Restart or update gateway; refresh connection credentials
Notebook activity failed	Exception inside the notebook	Open the snapshot of the run to see the failing cell
ForEach very slow	Sequential on, or low batch count	Turn off sequential; raise batch count
Activity timed out	Timeout shorter than workload	Raise timeout; optimise the workload

EXAM TIP Rerun from the failed activity to avoid repeating successful upstream steps.

REMEMBER Click the error icon on an activity run to read the full error code and message.

Diagnosing Dataflow Gen2 failures

Credentials or gateway

Refresh fails at the source step. Enter the connection credentials again; check gateway is online and has the source.

Data type errors

Cells show Error after a type change. Use Keep errors to inspect, then Replace errors or fix the source.

Destination schema mismatch

Columns no longer match the existing table. Update column mapping, or use dynamic schema with Replace.

Timeouts and slowness

Folding broke, so all rows load into the engine. Reorder steps so folding steps come first; enable staging or fast copy.

Where to look Refresh history, then the failed refresh, then per query and per activity details with the error message and duration.

Diagnosing notebook and Spark failures

Error	Cause	Fix
AnalysisException: table or view not found	No default lakehouse, or wrong name	Attach the lakehouse; use lakehouse.table names
OutOfMemoryError, executor lost	collect() on big data, skew, small nodes	Avoid collect; repartition; larger nodes
ModuleNotFoundError	Library not installed in the environment	Add it to the environment and publish
Schema mismatch on write	New or changed columns	mergeSchema option, or fix types
Session could not start or was rejected	Capacity busy or throttled	Retry later; scale; reduce concurrency
Concurrent update conflict	Two writers changed the same Delta files	Serialise writers or partition the writes

EXAM TIP For a scheduled notebook, open the run snapshot: it shows every cell’s output exactly as it ran.

EXAM TRAP Driver logs hold Python tracebacks; executor logs hold task failures. Read the right one.

Diagnosing Eventhouse problems

```
.show ingestion failures
| where FailedOn > ago(1d)
| project FailedOn, Table, Details, ErrorCode
```

```
.show queries
| where StartedOn > ago(1h) and State == "Failed"

.show table Telemetry policy update
```

Mapping errors: JSON paths do not match; columns arrive empty. Fix the ingestion mapping.

Update policy failures: schema mismatch between function output and target table.

Query limits: result set too large or memory exceeded. Filter on time first, summarize, use take.

Data missing: retention policy deleted it, or it is outside the hot cache and slow.

EXAM TIP Rows that “disappear” after a few weeks usually mean the retention policy soft delete period has passed.

Diagnosing Eventstream problems

Symptom	Likely cause	Resolution
Source shows an error, no events	Expired key or wrong connection string	Update the connection credentials
Events missing or split with another app	Shared Event Hubs consumer group	Give Eventstream its own consumer group
Destination rejects data	Format or schema mismatch with the table	Match JSON, CSV or Avro setting; fix mapping
Changes have no effect	Edits not published	Publish the Eventstream from edit mode
Latency rising, backlog growing	Throughput level too low for event rate	Raise the throughput setting; simplify operators
Aggregates look wrong	Wrong window type or timestamp field	Use event time; check window size and hop

REMEMBER Select any node, then Data insights for event counts and Runtime logs for warnings and errors.

EXAM TIP Use Data preview on the source node to confirm the payload shape before blaming the destination.

Diagnosing T SQL errors in Fabric

DML on a SQL analytics endpoint

INSERT, UPDATE or DELETE fails because the endpoint is read only. Write with Spark or move the table to a Warehouse.

Update conflicts

Snapshot isolation aborts one of two transactions updating the same table at once. Serialise writers or retry.

Unsupported data types

Some SQL Server types such as nvarchar, datetime and money are not supported. Use varchar, datetime2 and decimal.

Permission denied

The user has Read but no ReadData or object GRANT, or a DENY applies. Check item permissions, then GRANTs.

Invalid object name

Wrong schema or item name in a cross database query. Use Item.schema.table and check the item is in the workspace.

Conversion failed

Text cannot convert to a number or date. Use TRY_CAST or TRY_CONVERT and fix the source rows.

EXAM TIP Check the T SQL surface area documentation when a familiar SQL Server feature errors; the Warehouse supports a subset.

Diagnosing broken shortcuts

Target moved, renamed or deleted: the shortcut points nowhere. Recreate it with the new path.

403 on an internal shortcut: the reader lacks access to the target item. Grant it there.

403 on an external shortcut: rotated key, expired SAS or revoked service principal. Update the connection.

Shows as Unidentified under Tables: target is not a Delta table. Move it to Files or convert to Delta.

Network blocked: storage firewall. Use trusted workspace access or a gateway.

Slow or costly reads: cross cloud egress. Enable shortcut caching.

Diagnostic order Does the target exist? Can the identity reach it? Is the network open? Is the format right for the area it lives in?

Test yourself: monitoring and errors

Q1. Reports were slow every morning at 08:00. Which tool shows which items consumed capacity then?

Answer: Fabric Capacity Metrics app, timepoint drill through.

Q2. An Eventstream and a legacy app read the same Event Hub and both miss events.

Answer: Give each reader its own consumer group.

Q3. An UPDATE against a lakehouse table from the SQL endpoint fails. Why?

Answer: The SQL analytics endpoint is read only; use Spark or a Warehouse.

Q4. Analysts get 403 reading an internal shortcut that works for the owner.

Answer: Grant the analysts access to the target item; internal shortcuts pass their identity through.

DOMAIN 3 · OPTIMIZE PERFORMANCE

Where performance is won or lost

Lakehouse tables

File sizes, V Order, OPTIMIZE, VACUUM, partitioning, clustering

Pipelines

Parallelism, throughput, staging, fewer activities

Warehouse

Statistics, data types, clustering, caching

Eventstream and Eventhouse

Throughput, caching, retention, partitioning, batching

Spark

Joins, shuffles, skew, caching, native engine, profiles

Queries

Pruning, pushdown, Direct Lake, efficient KQL

REMEMBER Every optimisation also saves capacity units. Faster usually means cheaper in Fabric.

The small file problem

Before: 4,000 files of 250 KB



Each square stands for 100 files. Every file costs an open, a metadata read and a task.

After OPTIMIZE: 1 GB target files



Same 1 GB of data in roughly one large file: far fewer opens and tasks.

EXAM TIP Frequent small appends from streaming or trickle loads are the classic cause. Compact on a schedule.

REMEMBER Target file sizes are commonly in the hundreds of MB to about 1 GB.

V Order and optimized write

V Order

A write time optimisation of Parquet (sorting, row group distribution, encoding) that makes reads by Direct Lake, SQL and Spark much faster. Files stay standard Parquet.

Costs extra time on write; pays back on read heavy tables.

Optimized write

Spark shuffles data before writing so each partition produces fewer, larger files. Reduces the small file problem at source.

Ideal for partitioned tables and frequent appends.

```
spark.conf.set("spark.sql.parquet.vorder.default", "true")    # session level
ALTER TABLE gold.sales SET TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true")  # table level
spark.conf.set("spark.microsoft.delta.optimizeWrite.enabled", "true")
```

EXAM TRAP Newer workspaces may default to a write optimised profile with V Order off. Turn it on for Gold tables that Power BI reads.

OPTIMIZE, Z Order and VACUUM

```
OPTIMIZE silver.orders;
OPTIMIZE silver.orders WHERE OrderYear = 2026;
```

Bin compaction: rewrites many small files into fewer large ones. Old files stay until VACUUM.

```
OPTIMIZE silver.orders ZORDER BY (CustomerID);
OPTIMIZE silver.orders VORDER;
```

Z Order colocates related values so filters on CustomerID skip files. **VORDER** applies V Order while compacting.

```
VACUUM silver.orders RETAIN 168 HOURS;
DeltaTable.forName(spark, "silver.orders").vacuum(168)
```

VACUUM deletes files no longer referenced and older than the retention. Default 7 days (168 hours).

EXAM TRAP VACUUM with a short retention breaks time travel to older versions and can break long running readers.

EXAM TIP Order of operations: OPTIMIZE first, then VACUUM to reclaim storage.

Partitioning and clustering: data skipping

Partitioning

Separate folders per value, such as OrderYear=2026. Queries filtering on the column read only matching folders.

OrderYear=2026 (read)

OrderYear=2025 (skipped)

OrderYear=2024 (skipped)

Rules of thumb

- Partition only large tables (hundreds of GB and up)
- Low cardinality columns used in filters, such as year or region
- Each partition should still hold about 1 GB or more
- Never partition on a high cardinality key like CustomerID
- For high cardinality filters, use Z Order or liquid clustering

```
df.write.format("delta").partitionBy("OrderYear", "Region").saveAsTable("gold.sales")
CREATE TABLE gold.events (...) USING DELTA CLUSTER BY (DeviceId);  # liquid clustering, supported runtimes
```

EXAM TRAP Over partitioning creates thousands of tiny files and slows everything down.

Ways to run table maintenance

Lakehouse UI

Right click a table, Maintenance: choose OPTIMIZE (with optional V Order) and VACUUM with a retention. No code.

Scheduled notebook

Loop over tables running OPTIMIZE and VACUUM; schedule weekly or after big loads.

REST API

The table maintenance job API runs the same job from pipelines or external tools.

Automatic

Optimized write and auto compaction table properties keep files healthy on every write.

```
for t in ["silver.orders", "silver.customers", "gold.sales"]:
  spark.sql(f"OPTIMIZE {t} VORDER")
  spark.sql(f"VACUUM {t} RETAIN 168 HOURS")
```

DOMAIN 3 · OPTIMIZE A PIPELINE

Making pipelines faster and cheaper

Load time for 100 tables (illustrative)



Shape only. Gains flatten once the source or capacity saturates.

- **Parallelise** independent work: ForEach batch count, parallel branches.
- **Copy tuning:** source partitioning, parallel copies, throughput optimisation.
- **Load incrementally** instead of full reloads.
- **Reuse Spark sessions** with high concurrency session tags.
- **Push filters** into the source query, not after the copy.

EXAM TIP ForEach batch count defaults to 20 and can be set up to 50 when Sequential is off.

Warehouse design choices that drive speed

Right sized data types

Smallest type that fits: INT not BIGINT, VARCHAR(50) not VARCHAR(8000), DATE not DATETIME2 when time is unused.

Star schema

Integer surrogate keys for joins; avoid joining on long strings.

Batch your writes

Large INSERT ... SELECT or COPY INTO, not row by row inserts that create many small files.

Write efficient queries

Select only needed columns, filter early, avoid functions on filtered columns.

REMEMBER The Warehouse handles compaction and file layout automatically, so your levers are design, statistics, clustering and query shape.

Statistics, clustering and caching

Statistics

```
CREATE STATISTICS st_CustomerID
ON dbo.FactSales (CustomerID)
WITH FULLSCAN;
UPDATE STATISTICS dbo.FactSales;
```

Created automatically by the engine; add or update manually after big loads to keep plans accurate.

Data clustering

```
CREATE TABLE dbo.Events (
  EventDate DATE, DeviceId INT, ...)
WITH (CLUSTER BY (EventDate, DeviceId));
```

Colocates rows by commonly filtered columns so queries skip files. Check availability on Microsoft Learn.

Caching

In memory and SSD cache warms automatically for hot data.

Result set caching returns repeated identical queries instantly when enabled.

EXAM TIP A query got slow right after a huge load: update statistics first, before redesigning anything.

Tuning the real time path

Eventstream

- Set the throughput level to match peak event rate
- Filter and project early to shrink payloads
- Use enough source partitions for parallel reads
- Prefer direct ingestion to Eventhouse when no transform is needed
- Move heavy enrichment into KQL update policies

Eventhouse

- Caching policy: keep the queried period hot
- Retention policy: delete what nobody needs
- Partitioning policy on a high cardinality key
- Materialized views for repeated aggregates
- Batching policy: trade latency against efficiency
- Minimum consumption for steady latency

EXAM TIP “Queries on the last 7 days are fast but older data is slow” is a caching policy question: extend the hot cache period.

Eventhouse policies in practice

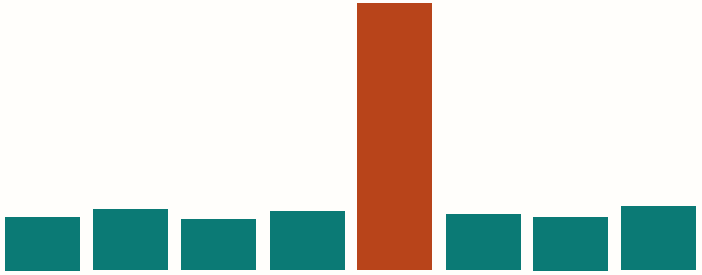
Policy	Controls	Example command
Caching	How long data stays in hot SSD cache	.alter table Telemetry policy caching hot = 30d
Retention	When data is soft deleted	.alter table Telemetry policy retention @'{"SoftDeletePeriod": "365.00:00:00"}'
Ingestion batching	Seal a batch by time, count or size (default 5 minutes, 1,000 items, 1 GB)	.alter table Telemetry policy ingestionbatching @'{"MaximumBatchingTimeSpan": "00:00:30"}'
Streaming ingestion	Seconds latency for small payloads	.alter table Telemetry policy streamingingestion enable
Partitioning	Groups extents by a key, speeding filtered queries	Set on DeviceId for many devices queried one at a time

EXAM TIP Lower the batching time span to cut ingestion latency when streaming ingestion is not an option.

EXAM TRAP A longer hot cache speeds queries but costs more. Match it to what users actually query.

Joins, shuffles and skew

Task durations in one stage (skewed)



One straggler (rust) holds the whole stage: a hot key.

Broadcast join small tables to avoid shuffling the big one.

Adaptive Query Execution (on by default) coalesces partitions and splits skewed ones at runtime.

Salting spreads a hot key across several partitions.

Filter and select early so less data reaches the shuffle.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
big.join(broadcast(small), "key")
```

More Spark levers

Resource profiles

Preset configs: writeHeavy for ingestion, readHeavyForSpark, readHeavyForPBI for Direct Lake tables.

Native execution engine

Vectorised execution of SQL and DataFrame operators, no code changes.

Cache wisely

df.cache() only for DataFrames reused several times; unpersist afterwards.

Partition count

Tune shuffle partitions to data size; repartition before wide writes, coalesce to reduce output files.

Avoid driver bottlenecks

No collect() or toPandas() on big data; avoid Python UDFs where built in functions exist.

Right sized pools

Memory heavy work on larger nodes; autoscale and dynamic allocation for variable loads.

```
spark.conf.set("spark.fabric.resourceProfile", "readHeavyForPBI")
spark.conf.set("spark.sql.shuffle.partitions", "auto")  # or a number sized to the data
```

EXAM TIP Built in Spark SQL functions beat Python UDFs because they run in the optimised engine, not row by row in Python.

Fast queries across every engine

Engine	Top techniques
Spark SQL	Partition pruning, predicate pushdown into Parquet, Z Order or clustering for selective filters, select only needed columns
T SQL	Up to date statistics, narrow types, filters on clustered columns, avoid SELECT *, check query insights for regressions
KQL	Time filter first, has over contains, summarize before join, small table on the left of join, materialized views
Direct Lake	V Order on Gold tables, fewer and larger files, avoid views that force DirectQuery fallback, stay within guardrails



EXAM TRAP In KQL, contains scans every string; has uses the term index. Use has for whole words.

CHECKPOINT 9 · OPTIMIZE PERFORMANCE

Test yourself: optimisation scenarios

Q1. A streaming load created 50,000 small files; Direct Lake reports are slow.

Answer: OPTIMIZE with V Order on the table, then schedule regular maintenance.

Q2. Storage keeps growing after daily MERGE operations.

Answer: Run VACUUM to remove unreferenced old files beyond retention.

Q3. One Spark stage has a single task running 20 times longer than others.

Answer: Data skew: enable AQE skew join handling or salt the hot key.

Q4. KQL queries over the last 90 days are slow but last week is fast.

Answer: Extend the caching policy hot period to cover 90 days.

FINAL SECTION

Exam readiness

Strategy, top traps, cheat sheets, glossary and your final checklist.



Strategy

Cheat sheets

Checklist

EXAM READINESS · STRATEGY

Managing the 100 minutes

Case study 25	Main section, first pass 60	Review 15

Suggested minutes; case study position varies by exam form.

Read the last line first. Know what is asked before reading the scenario.

Underline constraints: least effort, least privilege, no code, minimise cost, real time.

Eliminate first. Remove options that break a stated constraint, then choose.

Flag and move on. Never spend more than 2 minutes on one item at first pass.

Answer everything. There is no penalty for a wrong answer.

Microsoft Learn may be available during the exam; use it only for quick look ups.

EXAM READINESS · CASE STUDIES

Cracking a case study

1

Skim the questions

Know what to look for before reading tabs.

2

Map the estate

Sources, workspaces, teams, skills, capacity.

3

List requirements

Business, technical, security; note every “must”.

4

Answer per requirement

Each answer should satisfy a named constraint.

Signal words

“Minimise development effort” · “least privilege” · “near real time” · “without duplicating data” · “existing SQL skills” · “minimise cost”

Common answers they unlock

Copy job or mirroring · Viewer or item sharing · Eventstream and Eventhouse · shortcuts · Warehouse and T SQL · smaller SKU plus optimisation

EXAM TIP Case study tabs stay available for every question in the block, so revisit them instead of relying on memory.

EXAM READINESS · TOP TRAPS

Twelve traps that cost candidates marks

1 · The SQL analytics endpoint is read only

2 · ReadAll bypasses SQL row level security

3 · Contributors and above bypass RLS, masking and OneLake roles

4 · Masking is not encryption

5 · Git and deployment pipelines move metadata, not data

6 · Renaming items breaks deployment pairing

7 · Watermarks never catch hard deletes

8 · MERGE fails on duplicate source keys

9 · Non Delta data under Tables shows as Unidentified

10 · Streaming aggregation in append mode needs a watermark

11 · Short VACUUM retention breaks time travel

12 · Exiting a notebook marks the activity succeeded

EXAM READINESS · CHEAT SHEET 1

Requirement to Fabric feature

When the question says	Think
Use data in S3 or ADLS without copying	OneLake shortcut
Replicate an operational database continuously, no ETL	Mirroring
Incremental copy with least effort	Copy job
Hundreds of tables from a control table	Lookup plus ForEach plus parameterised Copy
Run when a file lands	Storage or OneLake event trigger
Per stage connection values	Variable library or deployment rules
Telemetry at seconds latency	Eventstream into Eventhouse, KQL
Transform rows as they arrive in KQL	Update policy
Alert on a streaming condition	Activator
Faster Spark with no code change	Native execution engine

EXAM READINESS · CHEAT SHEET 2

Syntax you must recognise instantly

PySpark and Delta

```
.merge(...).whenMatchedUpdateAll()
.whenNotMatchedInsertAll()
dropDuplicates(["id"])
withWatermark("ts", "10 minutes")
window(col("ts"), "5 minutes")
OPTIMIZE t ZORDER BY (c)
VACUUM t RETAIN 168 HOURS
```

T SQL

```
CREATE SECURITY POLICY ...
ADD FILTER PREDICATE f(col) ON t
MASKED WITH (FUNCTION = 'email()')
GRANT UNMASK TO [g]
GRANT SELECT ON t (c1, c2) TO [g]
COPY INTO t FROM '...'
CREATE TABLE t AS SELECT ...
```

KQL and expressions

```
summarize count() by bin(ts, 5m)
summarize arg_max(ts, *) by id
.alter table T policy caching hot = 30d
.show ingestion failures
@activity('A').output.firstRow.X
@item().Name
@pipeline().parameters.P
```

REMEMBER Code completion items usually hinge on one keyword: firstRow, arg_max, FILTER PREDICATE, UNMASK, ZORDER, checkpointLocation.

Glossary: A to L

Activator: no code engine that detects conditions in data and triggers actions.

Change data feed: Delta feature that records row level changes between versions.

Deployment pipeline: promotes content between stage workspaces.

Domain: logical grouping of workspaces by business area.

Environment: item packaging Spark runtime, libraries and properties.

Eventstream: no code item that ingests, transforms and routes events.

Capacity unit (CU): the measure of Fabric compute an F SKU provides.

DACPAC: compiled database project used for declarative schema deployment.

Direct Lake: semantic model mode reading Delta files straight into memory.

Endorsement: Promoted, Certified or Master data badge on an item.

Eventhouse: container for KQL databases optimised for time series.

Lakehouse: item combining Delta tables, files and a SQL analytics endpoint.

Glossary: M to Z

Materialized view: KQL aggregate kept current in the background.

Mirroring: continuous replication of a database into OneLake Delta tables.

OneLake security: roles on data enforced across Spark, SQL and Power BI.

Smoothing: spreading capacity usage over time to absorb spikes.

Update policy: KQL function that transforms data on ingestion.

Variable library: per stage values consumed by items.

Medallion: Bronze, Silver, Gold layering of data quality.

OneLake: the single tenant wide data lake behind every Fabric item.

Shortcut: pointer to data in OneLake or external storage, no copy.

Starter pool: always warm Spark pool for fast session start.

V Order: Parquet write optimisation for faster reads.

Watermark: last loaded value, or a lateness bound in streaming.

EXAM READINESS · FINAL CHECKLIST

Are you ready? Tick every box

- [ ] I can configure Spark, domain, OneLake and Airflow settings
- [ ] I can build a deployment pipeline with rules
- [ ] I can create OneLake security roles
- [ ] I can write dynamic expressions and triggers
- [ ] I can create shortcuts and mirrored databases
- [ ] I can build an Eventstream and pick a window type
- [ ] I can find failures in the Monitor hub and logs
- [ ] I can tune Spark, Warehouse and Eventhouse
- [ ] I can connect Git, branch out and resolve conflicts
- [ ] I can write RLS, CLS, OLS and masking in T SQL
- [ ] I can choose Dataflow Gen2, pipeline or notebook
- [ ] I can implement watermarks, MERGE and SCD Type 2
- [ ] I can aggregate and deduplicate in PySpark, SQL, KQL
- [ ] I can write a structured streaming job with a checkpoint
- [ ] I can run OPTIMIZE, VACUUM and apply V Order
- [ ] **I passed the free Microsoft practice assessment**

YOU ARE READY

Go and earn the badge

Book the exam once you score comfortably above passing on practice tests, then renew each year with the free online assessment.



Official study guide

learn.microsoft.com, search “DP 700 study guide”

Practice

Free practice assessment and exam sandbox

Hands on

Fabric trial capacity and the Microsoft Learn labs